Solving the U-Shaped Assembly Line Balancing Problem Type-II Using a Genetic Algorithm

Project Final Report

EMU427 – Heuristic Methods for Optimization

Department of Industrial Engineering
Hacettepe University

Students

Tarık Buğra BİRİNCİ	2210469046
Pınar Ece PANK	2240469085
Mustafa Alp ULAŞ	2210469034
Firuze İpek YILDIRIM	2220469028
Beril YILDIZ	2230469107

Instructor: Prof. Çağrı KOÇ

December 2025
Ankara, Türkiye

Contents

1 Introduction

Assembly line balancing is a fundamental optimization problem in industrial engineering. It aims to distribute tasks across workstations by trying to maximize productivity and minimize idle time. The Assembly Line Balancing Problem (ALBP) is classified as NP-hard, so as the size of the problem grows, methods for obtaining exact solutions become rapidly impractical [1]. As a result, heuristic and metaheuristic approaches have been developed and adapted to obtain high-quality results with reasonable computational effort. U-shaped assembly lines have significant popularity for being an effective alternative to traditional straight lines in production. They allow workers to work two sides of the line, so it reduces distance and eases multi-tasking.[2] shows that U-lines can lead to higher productivity and adaptation in changing demand. Type-2 assembly line balancing can be preferred when the number of workstations is fixed. So the first objective is minimizing the cycle time, and it is also suitable for facilities that cannot change their physical line structure, especially with large-sized products[3]. Since the possibilities of assembly line problems get exponentially larger because of their combinatorial structure and growing search area, deterministic optimization approaches remain limited and become computationally infeasible for medium to large scale problems. As a consequence, Genetic Algorithms (GAs) have become an alternative because of their stochastic search methods. In our project, numerous studies we researched mostly highlight that GAs can generate well-performing solutions without the problem needing to be convex or linear as in classical LP problems, so this makes them appropriate for NP-hard manufacturing problems. The essential consideration for U-shaped assembly lines is that the encoding part has to represent bidirectional task assignments and preserve precedence relationships in the evolutionary process. This project aims to develop and analyze a GA-based approach specifically for Type-2 U-shaped balancing. Even though various metaheuristic approaches are used in classical assembly line balancing problems and show very strong performance in different combinations of the classical assembly line problem, there are a limited number of studies that specifically work on the performance of these algorithms and their applicability for U-shaped Type-2 line balancing. There is a limited number of encoding schemes that represent forward and back task assignments and preserve precedence constraints, while effectively minimizing cycle time in a fixed number of workstations specifically for this problem.

2 Problem Formulation - Detailed Problem Description (UALBP-2)

2.1 Introduction to Assembly Line Balancing

Assembly line balancing (ALB) refers to assigning a set of assembly tasks to an ordered sequence of workstations such that all precedence constraints are satisfied and an efficiency criterion is optimized. Each task has a processing time, and some tasks must be completed before others (precedence relations). In straight-line assembly systems, workstations are arranged linearly, and a task can only be assigned after all of its predecessors are allocated to earlier stations.

Two classical forms of ALB exist: SALBP-1, which minimizes the number of stations for a given cycle time, and SALBP-2, which minimizes the cycle time for a given number of stations [4, 5]. Both are NP-hard combinatorial problems [6].

2.2 Characteristics of U-Shaped Assembly Lines

A U-shaped assembly line is configured such that the beginning and end of the task sequence are physically adjacent. In this layout, a worker can operate on both sides of the U, giving access to tasks from the front (forward direction) or the back (reverse direction) of the precedence graph.

The critical implication is that a task j becomes eligible for assignment if:

- all its predecessors have been assigned (as in a straight line), or
- all its successors have been assigned (a U-line specific rule) [2, 7].

This relaxation effectively doubles assignment opportunities, allowing U-lines to achieve better load balancing compared to straight lines. Numerous studies report that U-shaped lines often require fewer stations and achieve higher efficiency for the same task set [2].

2.3 UALBP-2 Definition

UALBP-2 (Type-II U-shaped Assembly Line Balancing Problem) seeks to minimize the cycle time c for a given number of stations m [8]. The input consists of:

1. a set of n tasks with processing times t_i,
2. a precedence graph represented as a DAG,
3. a fixed number of stations m.

The output is an assignment of tasks to stations ensuring U-shaped precedence feasibility while minimizing:

$$c = \max_{j=1,\ldots,m} \left(\sum_{i \in S_j} t_i \right),$$

where S_j denotes the set of tasks assigned to station j.

2.4 Assumptions and Constraints

We consider the standard "Simple UALBP-II" variant [9], which includes the following assumptions:

- Single-model production: Only one product type is assembled.
- Deterministic task times: Processing times t_i are fixed and known.
- Single-sided work: Each worker operates on the inner side of the U.
- Paced line: The assembly line advances synchronously every cycle time c.
- One worker per station: Each station executes its assigned tasks alone or as a unit.
- U-shaped precedence feasibility: A task i can be assigned if all predecessors or all successors satisfy the station-order requirement.
- Fixed number of stations: m is given and constant.

Under these assumptions, UALBP-2 becomes a partitioning problem: partition n tasks into m ordered groups while minimizing the maximum station load.

2.5 Computational Complexity

UALBP-2 is computationally challenging. Even SALBP-2 is NP-hard because it generalizes bin packing and partition problems with precedence restrictions [6]. UALBP-2 expands the search space due to bidirectional task eligibility.

Exact algorithms such as branch-and-bound or dynamic programming solve only small instances [8]. Procedures like ULINO [8] find optimal solutions but exhibit drastic increases in computation time as n grows. Due to this difficulty and the scarcity of exact solutions for larger problem sizes, heuristic and metaheuristic methods such as genetic algorithms are widely recommended.

2.6 Importance of Solving UALBP-2

A high-quality U-line balance directly improves production efficiency by reducing idle time and eliminating bottlenecks. Solving UALBP-2 provides actionable insights for:

- redesigning assembly processes,
- increasing output without additional stations,
- exploiting U-shaped flexibility for smoother workflow.

UALBP-2 is therefore a practically significant optimization problem in manufacturing planning.

2.7 MILP Formulation for UALBP-2

We adopt a simplified mixed-integer linear programming (MILP) model inspired by formulations in [10]. The objective is to assign tasks to m stations while satisfying U-shaped precedence and minimizing cycle time.

Sets and Parameters

- $i = 1, \ldots, n$: tasks
- $j = 1, \ldots, m$: workstations
- $\mathcal{P}$: set of precedence pairs (p, q)
- $Pred(i)$: set of predecessors of task i
- $Succ(i)$: set of successors of task i
- t_i: processing time of task i
- M: sufficiently large constant ($M \geq m$)

Decision Variables

- $X_{ij} \in \{0, 1\}$: equals 1 if task i is assigned to station j
- $y_i \in \{0, 1\}$: 0 for forward feasibility, 1 for backward feasibility
- $S_i \in \{1, \ldots, m\}$: station index of task i
- $c \geq 0$: cycle time (to be minimized)

Objective Function

$$\min \ c \tag{1}$$

Constraints

1. Task Assignment Each task must be assigned to exactly one station:

$$\sum_{j=1}^{m} X_{ij} = 1 \qquad \forall i \tag{2}$$

2. Station Index Definition The station index of each task is defined as:

$$S_i = \sum_{j=1}^{m} j X_{ij} \qquad \forall i \tag{3}$$

3. Cycle Time Constraint The total processing time at each station cannot exceed the cycle time:

$$\sum_{i=1}^{n} t_i X_{ij} \leq c \qquad \forall j \tag{4}$$

4. U-Shaped Precedence Constraints *Forward feasibility* for each predecessor $p \in Pred(q)$:

$$S_p \leq S_q + M y_q \tag{4a}$$

Backward feasibility for each successor $s \in Succ(q)$:

$$S_s \leq S_q + M(1 - y_q) \tag{4b}$$

These constraints ensure that task q is feasible in either the forward or backward direction.

5. Variable Domains

$$X_{ij},\, y_i \in \{0, 1\}, \quad S_i \in \{1, \ldots, m\}, \quad c \geq 0 \tag{5}$$

3 Description of the Genetic Algorithm

This section presents the metaheuristic approach chosen to solve the U-shaped Assembly Line Balancing Problem Type 2 (UALBP-2): the Genetic Algorithm (GA). Genetic Algorithm is an evolutionary optimization method that takes its inspiration from the concepts of natural selection and genetics. It is particularly suitable for problems in the field of combinatorial optimization where the search space is enormous and discrete.
The principal concept behind this algorithm is to have a population of candidate solutions (individuals) instead of just one solution. An iterative process, which is similar to evolution, undergoes selection, crossover (recombination), and mutation operations. This, in turn, helps the search to not only simultaneously explore different regions of the solution space but also to realize the benefits of good-quality partial solutions. In the UALBP-2 case, the algorithm works on producing a sequence of tasks (permutation), which is then decoded to find out the minimum feasible cycle time for a certain number of stations (m).

3.1 The Algorithm

The framework of our Genetic Algorithm is outlined in algorithm 2. The core component of the algorithm is the representation of the problem. We use a permutation-based representation, where an individual is a topological sort of the tasks. The algorithm begins by initializing a random population of feasible task sequences. In every generation, the fitness of each individual is evaluated. Since we are solving UALBP-2 (minimizing cycle time c for fixed m), the evaluation function involves a sub-procedure that calculates the minimum possible cycle time for a given sequence. To generate new offspring, we employ Tournament Selection to choose parents. These parents undergo POX (Precedence Operation Crossover) to produce a child that inherits structural characteristics from both parents. To prevent premature convergence, Swap Mutation is applied with a low probability. Finally, because random genetic operations may violate precedence constraints, a Repair Mechanism is applied to ensure the child remains a valid topological sort.

For the algorithm, we utilized a dataset from assemblylinebalancing.de, a widely cited repository in academic literature that aggregates problem instances from numerous studies. We selected a specific instance from this collection for our analysis.

The file structure is organized as follows: the initial lines represent the duration of each task. These are followed by the predecedence constraints, which are delimited by commas. Finally, an end-of-file marker indicates the termination of the data.

Algorithm 2: Genetic Algorithm for UALBP-II

Require: Tasks T, task times t, precedence graph P, number of stations m, population size N_{pop}, generations N_{gen}, crossover rate P_c, mutation rate P_m
Ensure: Best cycle time $C_{\min}$ and best assignment/permutation *BestSol*

ALGORITHM: GENETIC_ALGORITHM_FOR_UALBP_II
BEGIN
// 1. Initialization
Population $\leftarrow$ [ ]
For $i \leftarrow 1$ **to** N_{pop} **do**
    Candidate $\leftarrow$ GENTOPOSORT(T, P)
    APPEND(*Population*, *Candidate*)
End For
BestSol $\leftarrow \emptyset$
BestFit $\leftarrow 0$
$C_{\min} \leftarrow +\infty$
// 2. Main Evolution Loop
For *Generation* $\leftarrow 1$ **to** N_{gen} **do**
    FitnessScores $\leftarrow$ [ ]
// Evaluation
    For all *Individual* $\in$ *Population* **do**
        CurrentC $\leftarrow$ DECODEASSIGN(*Individual*, m, t, P)
        Fitness $\leftarrow 1/$*CurrentC*
        APPEND(*FitnessScores*, *Fitness*)
        If *Fitness* $>$ *BestFit* **then**
            BestFit $\leftarrow$ *Fitness*
            BestSol $\leftarrow$ *Individual*
            $C_{\min} \leftarrow$ *CurrentC*
        End If
    End For
    NewPopulation $\leftarrow$ [ ]
    APPEND(*NewPopulation*, *BestSol*) ▷ elitism
    While $|$*NewPopulation*$| < N_{\text{pop}}$ **do**
        *Parent*1 $\leftarrow$ SELECT(*Population*, *FitnessScores*)
        *Parent*2 $\leftarrow$ SELECT(*Population*, *FitnessScores*)
        If RAND$(0,1) < P_c$ **then**
            Child $\leftarrow$ CROSSOVER(*Parent*1, *Parent*2)
        Else
            Child $\leftarrow$ *Parent*1
        End If
        If RAND$(0,1) < P_m$ **then**
            Child $\leftarrow$ MUTATE(*Child*)
        End If
        If VALIDPREC(*Child*, P) **then**

```
            Append(NewPopulation, Child)
        Else
            Child ← RepairTopo(Child, P)
            Append(NewPopulation, Child)
        End If
    End While
    Population ← NewPopulation
End For
Return (BestSol, C_min)
END
```

3.2 Implementation

The algorithm 2 has been developed using the programming language Python due to its excellent high-level data structure handling ability.
The complete implementation by Python is provided in Appendix section A.

3.2.1 Constructive Heuristic

When tackling the Type-2 U-Shaped Assembly Line Balancing Problem (UALBP-2), where the aim is to reduce the cycle time for a predetermined number of stations, a major difficulty is that of making the sequence feasible. One of the solutions proposed is to implement topological sorting-based constructive heuristic within the Genetic Algorithm, which draws upon the dependency graph to produce a precedence-feasible list of tasks. This topological ordering not only guides the decoding phase with its precedence constraints but also facilitates the search for the best cycle time by allowing task assignment at either end of the stations.

Algorithm 3 InitializePopulationTopological (Random Topological Sort)

Require: Population size $PopSize$, precedence graph $G = (V, E)$
Ensure: Initial population P

$P \leftarrow [\,]$
For $p \leftarrow 1$ **to** $PopSize$ **do**
    $\pi \leftarrow [\,]$
    $S \leftarrow \{v \in V \mid indeg(v) = 0\}$ ▷ Eligible nodes
    While $S \neq \emptyset$ **do**
        select u uniformly at random from S
        append u to π
        remove u from S and update indegrees of its successors
        add any newly zero-indegree nodes to S
    End While
    append π to P
End For
Return P

3.2.2 Chromosome Representation

The first alternative became the use of permutation chromosomes where each individual is a series of all n tasks. The order of the genes from left to right indicates the priority that will be applied in the decoding process. This encoding has become the most popular one in assembly-line balancing because it allows the genetic operators to perform the whole task order manipulation without resulting in a non-valid permutation.[11]. Nevertheless, a chromosome in UALBP-2 does not associate with the station assignment directly like in straight-line SALBP-2. On the contrary, the permutation gives an ordered list that the U-line decoding algorithm uses to create a feasible solution. The reason is that U-shaped lines allow the assignments to be done in both directions, thus the decoding phase evaluates the permutation in a manner that shows this unique bidirectional feasibility.[11].

3.2.3 U-Shaped Decoding

Given a task permutation, the decoding procedure assigns tasks to stations progressively from station 1 to station m. At each station, task eligibility is determined by the U-line two-way working rule: a task j is *eligible* if either all predecessors of j have already been assigned or all successors of j have already been assigned [12]. This rule reflects the fact that, in a U-shaped layout, tasks can be performed from both the forward and the backward directions.

Among the currently eligible tasks, those appearing earliest in the chromosome are considered first and are assigned to the current station as long as the cumulative station load does not exceed the current cycle-time estimate. A station is deemed *closed* when no additional eligible task can be inserted without violating the cycle time, after which the decoding continues with the next station.

To evaluate a permutation efficiently, the decoding method incorporates a binary search to determine the smallest cycle time that yields a feasible assignment for the given permutation. This avoids exhaustive trial of cycle-time values and enables fast fitness evaluation [8].
A task j is eligible if

$$\text{Pred}(j) \subseteq A \quad \text{or} \quad \text{Succ}(j) \subseteq A,$$

where A is the set of already assigned tasks, and $\text{Pred}(j)$ and $\text{Succ}(j)$ denote the predecessor and successor sets of task j, respectively.

3.2.4 Fitness Function

Taking into consideration that UALBP-2 is a Type-II problem, the aim is to minimize the cycle time c while the number of stations m is given. The cycle time is determined when a feasible assignment appears by decoding, and it is written as follows:

$$c = \max_{k=1,\dots,m} \sum_{j \in S_k} t_j, \tag{6}$$

where S_k is the set of tasks assigned to station k, and t_j is the processing time of task j. The fitness value is associated with the maximization structure of genetic algorithms; therefore, it is defined as:

$$\text{Fitness} = \frac{1}{c}. \tag{7}$$

Therefore, solutions with smaller cycle times are awarded higher fitness values. On the other hand, if a permutation does not lead to any feasible assignment for any cycle time (i.e., decoding fails), then its fitness is set to 0, penalizing such individuals as infeasible. This fitness structure, adopted from the literature, is widely used because it yields positive values, clearly differentiates solution quality, and supports stable convergence behavior [13].

3.2.5 Parent Selection: Tournament Selection ($k = 2$)

The selection of parents is done by using the tournament selection method of size 2.This has been found efficient for combinatorial optimization problems where the fitness landscape includes many plateaus. The strategy of selection by tournaments is regarded as highly effective for combinatorial optimization problems that have tough fitness landscapes with many plateaus. Tournament selection not only preserves genetic diversity but also avoids a common problem seen in roulette-wheel selection, which is the sensitivity to scaling. [14]

Algorithm 4 TournamentSelection ($k = 2$)

Require: Population P, fitness array F, tournament size $k = 2$
Ensure: Selected parent permutation

$BestFit \leftarrow -\infty$
$BestIndex \leftarrow -1$
For $r \leftarrow 1$ **to** k **do**
    $i \leftarrow$ random index in $\{1, \ldots, |P|\}$
    If $F[i] > BestFit$ **then**
        $BestFit \leftarrow F[i]$
        $BestIndex \leftarrow i$
    End If
End For
Return $P[BestIndex]$

3.2.6 Crossover Operator: Precedence-Preserving Order Crossover (POX-like)

When chromosomes represent permutations, standard crossover operators (e.g., one-point or two-point crossover) may easily introduce infeasibility and duplicated tasks. Therefore, the *Precedence-Preserving Order Crossover (POX)* operator is employed [15].
The POX operator proceeds as follows:

1. A random subset of tasks is selected.
2. The offspring chromosome is initialized as an empty sequence.
3. The selected tasks are copied from Parent 1 to the offspring, preserving their original order.
4. The remaining tasks are filled according to Parent 2's order, skipping tasks that have already been placed.

This procedure preserves major subsequences from both parents and typically reduces the need for extensive repair operations. POX is particularly suitable for UALBP-2 because tight precedence constraints make it crucial to keep coherent task blocks intact.

Algorithm 5 POXCrossover (Precedence-Preserving Order Crossover)

Require: Parent permutations P_1, P_2
Ensure: Child permutation *Child*

$ChosenTasks \leftarrow \emptyset$
For all task x in P_1 **do**
    If RANDOM < 0.5 **then**
        add x to $ChosenTasks$
    End If
End For
If $ChosenTasks = \emptyset$ **then**
    add one random task from P_1 to $ChosenTasks$
End If
$Child \leftarrow [\,]$
For all task x in P_1 **do**
    If $x \in ChosenTasks$ **then**
        append x to $Child$
    End If
End For
For all task y in P_2 **do**
    If $y \notin ChosenTasks$ **then**
        append y to $Child$
    End If
End For
Return *Child*

3.2.7 Mutation Operator: Swap Mutation

Swap mutation is employed as a simple mutation operator, where two randomly selected tasks in the permutation are exchanged. This operator introduces small but effective perturbations, preserving most of the chromosome structure while preventing premature stagnation of the population. Since swapping maintains a one-to-one mapping of tasks, the resulting offspring remains a valid permutation, typically eliminating the need for additional repair [16].

In permutation-based genetic algorithms, mutation is generally applied with a low probability (e.g., 0.05–0.10) to balance exploration and stability [17].

Algorithm 6 SwapMutation

Require: Permutation *Perm*
Ensure: Mutated permutation *Perm*

$n \leftarrow |Perm|$
choose distinct indices i, j uniformly at random from $\{1, \ldots, n\}$
swap $Perm[i]$ and $Perm[j]$
Return *Perm*

3.2.8 Repair Mechanism

After crossover and mutation, a lightweight repair operator is applied to every permutation. If a task is found to violate precedence by appearing before any of its predecessors, it is gradually shifted to the right until the violation is removed. This procedure restores a valid topological order while preserving the original permutation structure as much as possible.

Algorithm 7 RepairToTopological (Lightweight Repair)

Require: Permutation π, precedence graph $G = (V, E)$
Ensure: Repaired permutation π

build an array $pos(\cdot)$ giving the index of each task in π
$changed \leftarrow$ **true**
While *changed* **do**
    $changed \leftarrow$ **false**
    For all $(u \rightarrow v) \in E$ **do**
        If $pos(u) > pos(v)$ **then** ▷ precedence violation
            remove v from its current position in π
            insert v immediately after u in π ▷ shift right
            update $pos(\cdot)$
            $changed \leftarrow$ **true**
        End If
    End For
End While
Return π

3.2.9 Termination Criteria

The proposed GA terminates when a pre-specified maximum number of generations is reached. In this study, the generation limit is set to 300, which is consistent with common practice in the assembly line balancing literature. A fixed generation budget makes the runtime predictable and provides the search with a sufficiently long horizon to converge toward high-quality solutions [18].

4 Results

In this part, the paper showcases the outcomes derived from the Genetic Algorithm (GA) that was utilized for solving the ARC83 U-shaped Assembly Line Balancing Problem Type II (UALBP-2) benchmark instance. The results are based on several CSV outputs generated by the implementation, including global summaries, run-by-run statistics, detailed descriptions of the best solution, Top-10 individuals of the best run, and parameter tuning logs.

4.1 Parameter Tuning

U-shaped assembly lines optimization involves a nonlinear search space that results from the two-entry task assignment logic. This report describes the two-phase fine-tuning approach. Phase I focuses on the stochastic operators, namely Mutation and Crossover, while the computational intensity scaling is measured in terms of Population and Generations in Phase II. Findings suggest that the high-intensity mutation approach together with the moderate population size produce the most effective Pareto-optimal solution.
To analyze the trade-off between exploration and exploitation, the computational budget was set to a fixed value ($P = 50$, $G = 250$) and the crossover rate parameter ($CR \in [0.2, 0.9]$) and mutation rate parameter ($MR \in [0.01, 0.8]$) were changed.

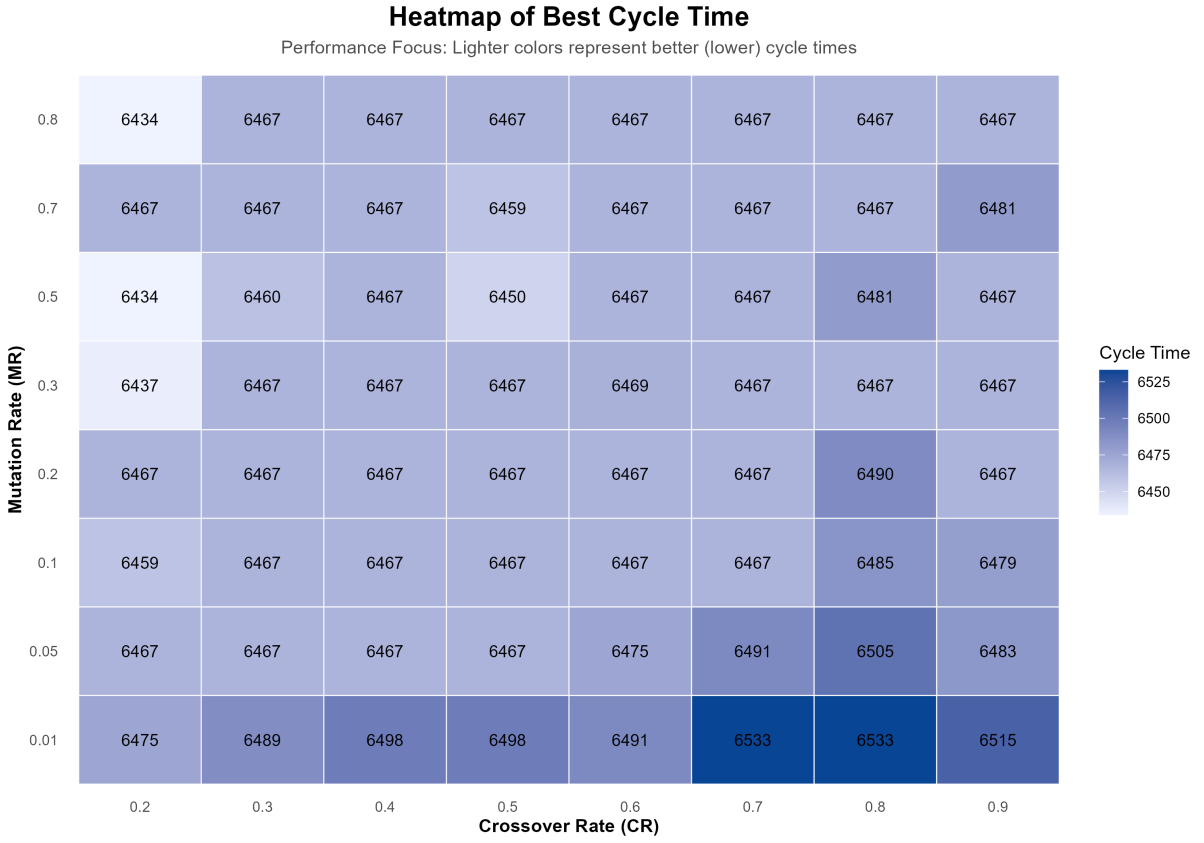

Figure 1: Cycle Time Heatmap

The timeline heatmap results show the so-called "Energy Gradient," with execution time rising diagonally from bottom left to top right. Within the UALBP-2 problem setting, POX crossover and swap mutation gene transfer often results in offspring violating precedence constraints. The marked increase in execution time, which reached 134.2 s for $CR = 0.9$, $MR = 0.8$, is solely due to CPU cycles utilized by the `repair_to_topological` function.

According to conventional literature, the rate of mutation must lie between 1% and 10%. However, it often is insufficient in cases of highly constrained discrete search problems, as in assembly line problems. The scientific rationale for having a relatively high value of MR (up to 0.5) in the study is the relative complexity of U-shaped lines. This warrants having strong strategies to get rid of the local optimum. For cases such as ARC83, having a high density of precedence constraints, low mutation rate results in a phenomenal decay of diversity, which results in the stagnation of chromosomes prior to the identification of a more efficient station load. The modern literature of heuristics pervasively favors the use of high mutation in NP-hard problems as a crucial "diversification mechanism" of the GA. It makes it possible to explore the unvisited area of the solution space. Hence, the 0.5 rate ensures the conservative role of the POX operator.

In contrast to linear balancing (SALBP), U-shaped balancing allows more permutations. From the performance heatmap above, low mutation rates ($MR = 0.01$) are associated with high cycle times (approximately 6533 s). On the other hand, higher mutation rates enable the search to explore "back-end" task assignments that would otherwise have been ignored by a generic greedy tactic. A better mutation rate and CR of 0.5 and 0.2, respectively, are used to investigate the scalability of the search process.

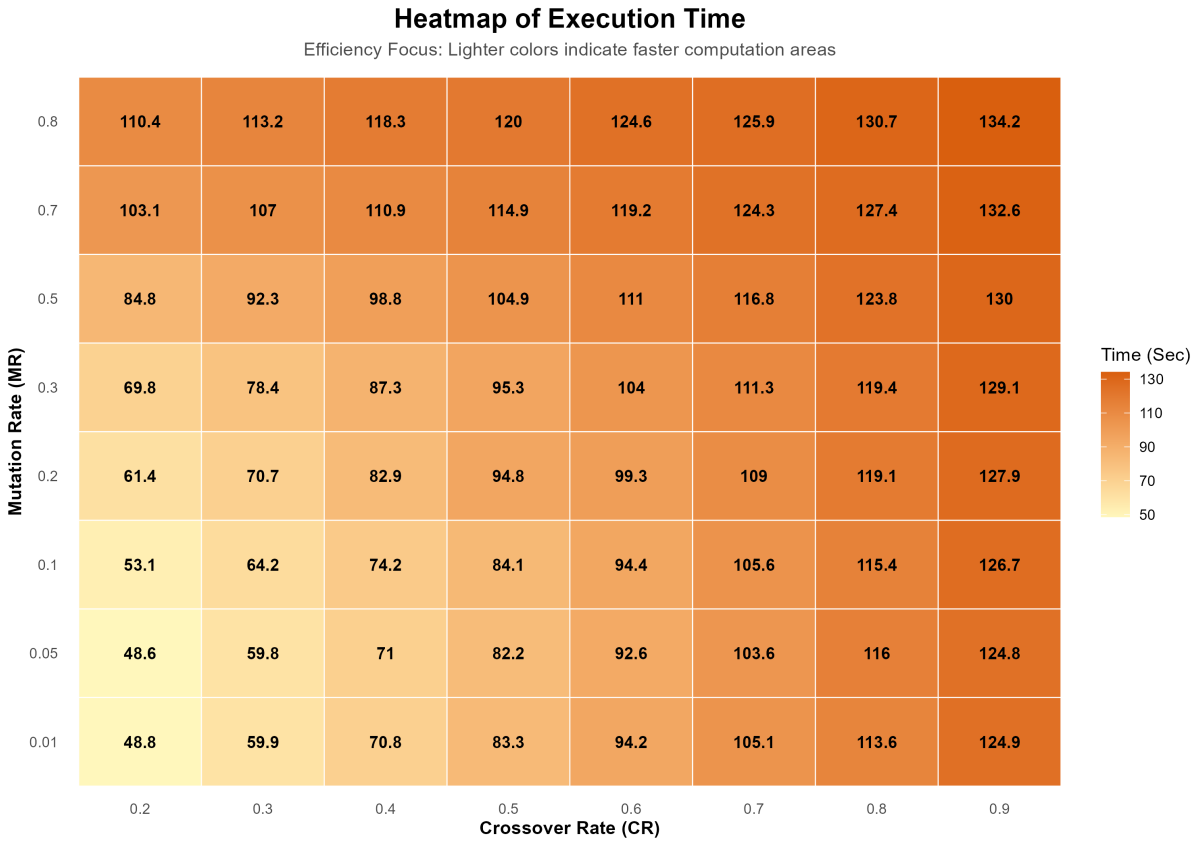

Figure 2: Execution Time Heatmap

Computational trade-off analysis results identify the bound of efficiency (Pareto frontier). Crossing P: 50 to P: 100 generates a sharp increase in the objective function; in contrast, crossing P: 100 to P: 150 generates a linear increase. $P = 150/G = 750$ resulted in the absolute best cycle time of 6425 s, with $P = 100/G = 250$ achieving 6438 s. A 0.2% cycle time improvement with a 141% increase in execution time (74.9 s to 180.5 s) is necessary for optimization of cycle times. Iterative design and real-time manufacturing optimization require engineering parameter $P = 100$ to be chosen predominantly.

An important aspect in Pareto charts is the size of the standard deviation (SD) error bars. SD at $P = 50$ (18.97) symbolizes a high degree of variability, signifying a population size that is too small in relation to the search space with a tendency to produce "hit-and-miss" solutions. SD at $P = 100$ (9.17) symbolizes the maximum level of stability, signifying that population size is appropriate in relation to the complexity of the 83-task ARC83 problem. SD at $P = 150$ (17.20), paradoxically, now tends to be even higher due to the large search space that enables multiple runs to settle on varied but promising areas on the Pareto frontier.

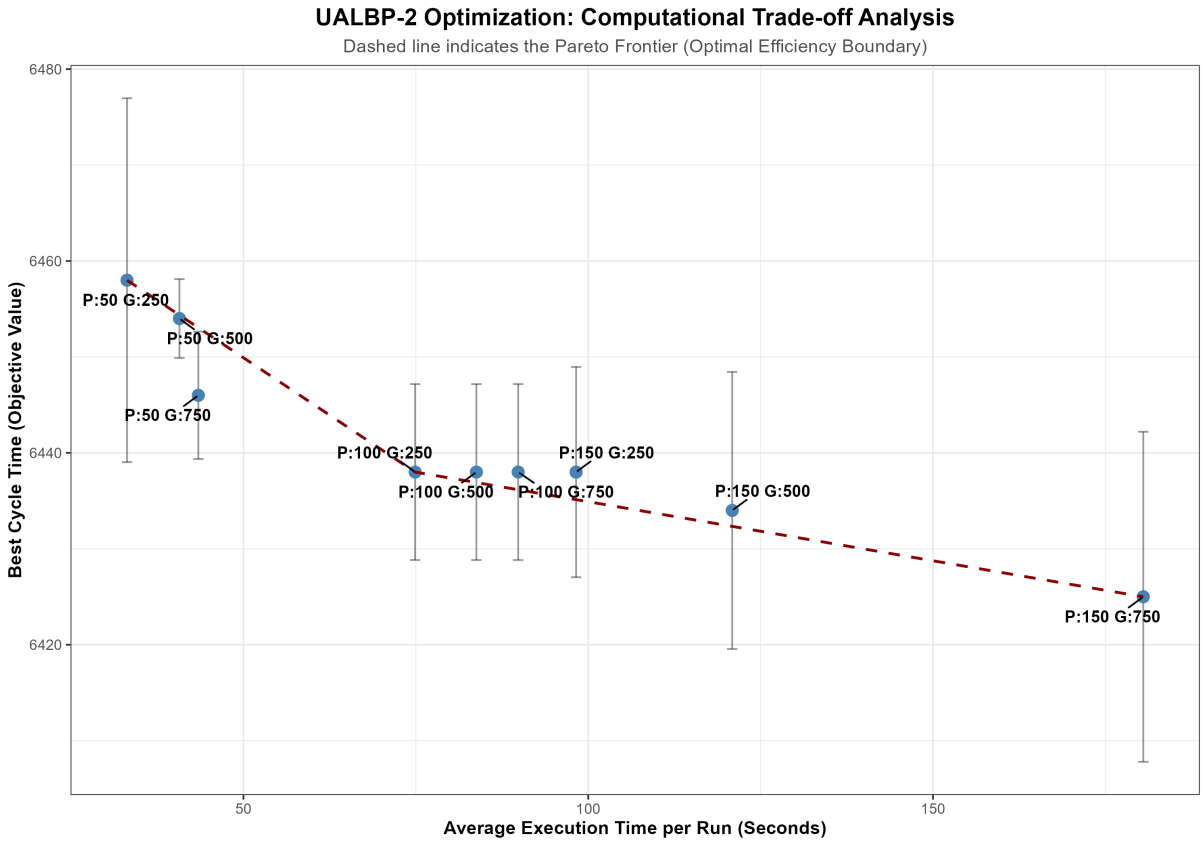

Figure 3: Execution Time Heatmap

The suggested setup gives a cycle time of 6438 s with a high level of reliability and an execution time under 80 s. From the results, a mutation rate of 0.5 is suggested for dealing with the dual-ended task problem, a crossover rate of 0.2 for sustaining the sequences, a population size of 100 for optimal robustness, and a generation limit of 250 for obtaining convergence without diminishing returns. Tournament selection with $k = 2$ is suggested for appropriate selection pressure.

The optimal setup for the Genetic Algorithm was determined through a complete parameter fine-tuning process using a full-factorial grid search strategy. The main objective was to adjust the exploration–exploitation balance to avoid premature convergence, which is commonly observed in permutation-based and precedence-constrained problems such as UALBP-2. Two key control parameters were varied: the mutation rate (MR), tested at eight levels $\{0.005, 0.01, 0.02, 0.04, 0.06, 0.08, 0.12, 0.20\}$, and the crossover rate (CR), tested at eight levels $\{0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.90, 0.98\}$. This resulted in $8 \times 8 = 64$ parameter combinations. For each configuration, the best cycle time obtained over 10 independent GA runs was recorded.

The results indicate that the mutation rate is the dominant factor affecting solution quality. Very low mutation (MR = 0.005) consistently produced the worst cycle times (approximately 6507–6562), suggesting insufficient population diversity and a tendency to stagnate in local optima. Likewise, MR = 0.01 still led to relatively weak performance

(about 6508–6558). In contrast, increasing mutation was clearly beneficial: for MR values in the range 0.04–0.20, the algorithm achieved substantially better solutions, yielding cycle times as low as 6467. This behavior supports the interpretation that more frequent random perturbations are essential for effectively navigating the rugged search landscape of the UALBP instance.

Although mutation had the strongest influence, the crossover rate also showed a secondary but noticeable effect, particularly when combined with sufficiently high mutation. The best-performing region was observed where MR was moderate-to-high (e.g., 0.04–0.20). In this interval, relatively lower crossover values (CR = 0.55–0.70) produced the best results, repeatedly reaching the minimum cycle time of 6467. For example, the combination MR = 0.08 and CR = 0.70 attained this minimum value, whereas increasing CR to 0.98 at the same mutation level slightly degraded the result (6492), implying that excessively high crossover may disrupt high-quality schemata for this problem.

Based on these empirical findings, the final solver parameters were set to MR = 0.08 and CR = 0.70. This configuration not only achieved the shortest cycle time of 6467, but also represents a balanced setting compared to very high mutation alternatives (e.g., MR = 0.20), which may introduce excessive randomness. Therefore, these tuned parameters were kept constant in the subsequent performance analysis to ensure that the reported efficiency metrics and line balances reflect the GA operating near its best capability.

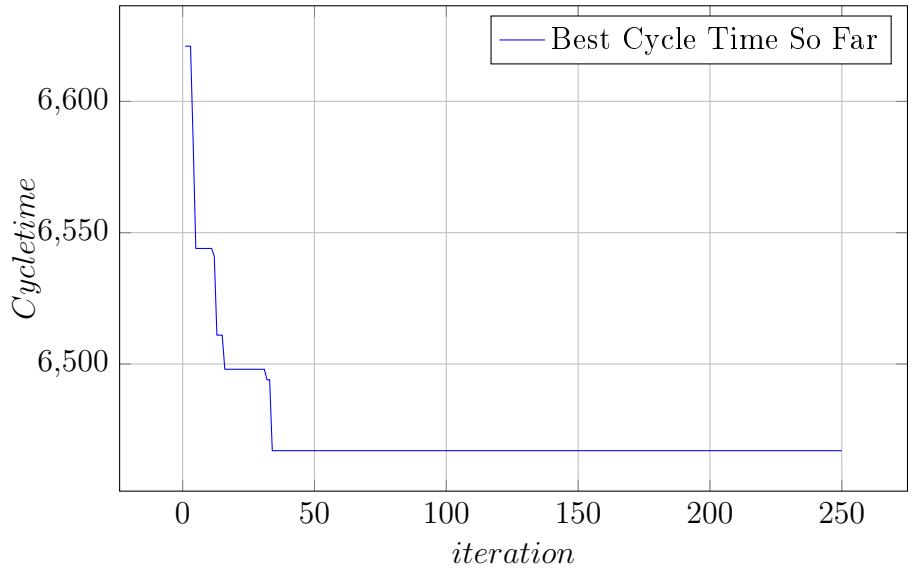

Figure 4: Convergence graph for ARC83, Run 1

As seen in figure 4 and Bkz. **??**., the best-so-far cycle time decreases rapidly during the early iterations, followed by stepwise improvements characterized by long plateaus and occasional sudden drops when better solutions are found. Toward the later iterations, improvements become marginal and infrequent, indicating that the genetic algorithm converges to near-stationary, similar-quality solutions for the ARC83 instance.

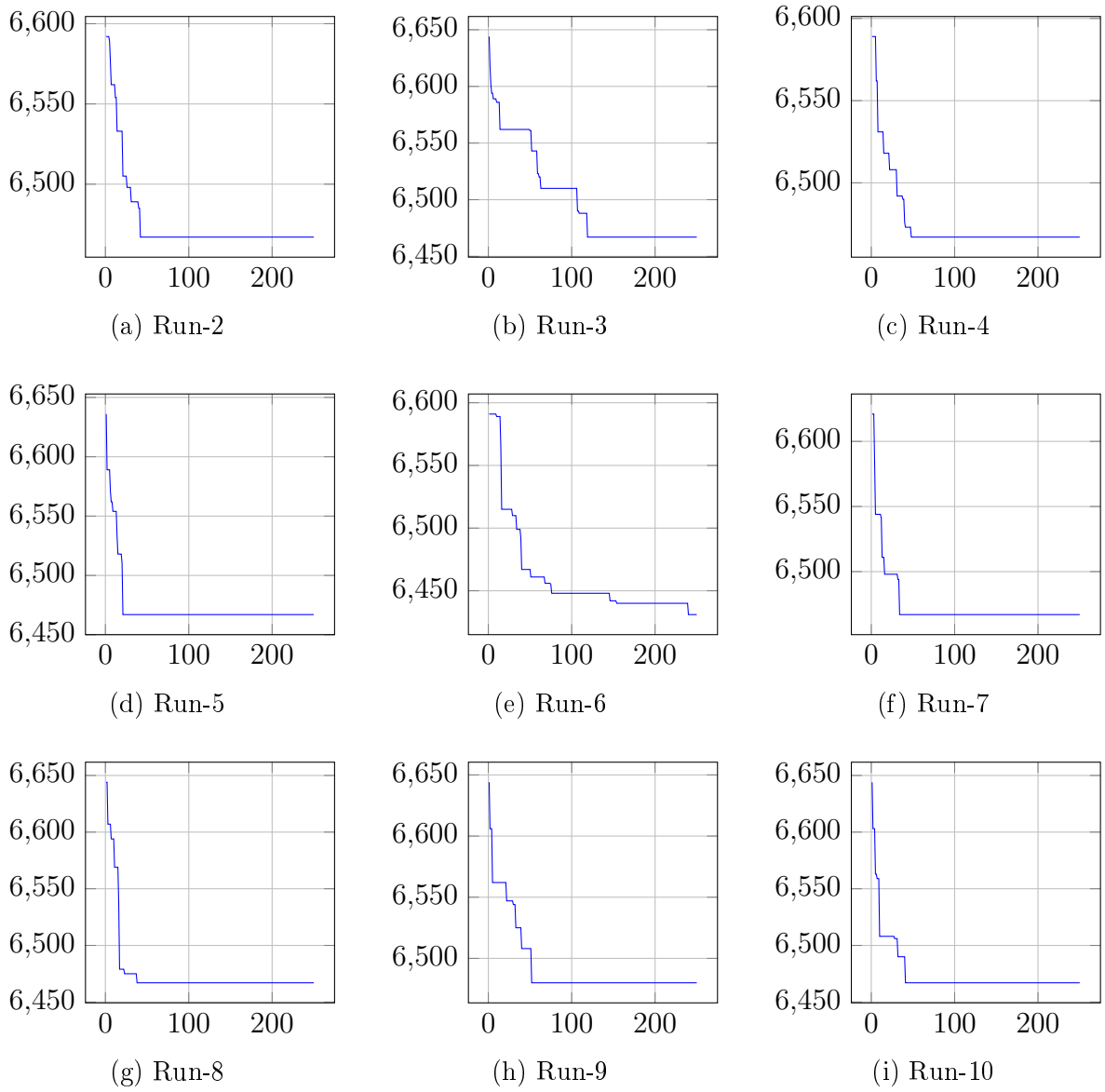

Figure 5: Convergence graphs for ARC83 (Best cycle time so far)

4.2 GA Parameter Settings

The final parameter settings, which were in use during all the experiments, are written below as per GA_summary_results.csv:

- Population size: 40
- Generations: 300
- Crossover rate: 0.7
- Mutation rate: 0.08
- Selection method: Tournament (k = 2)

- Crossover method: POX (precedence-preserving)
- Mutation method: Swap
- Number of independent runs: 10
- Number of stations vary for instances.

4.3 Run-by-Run Analysis

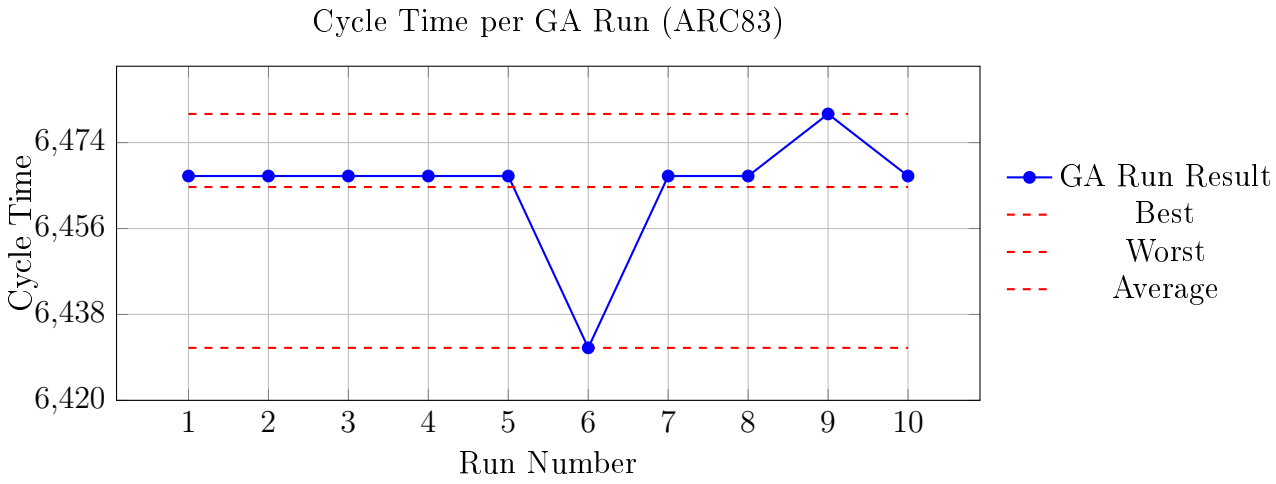

Figure 6: Instance ARC83 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

Detailed run-by-run best cycle times and runtimes for each benchmark instance (10 independent runs) are reported in Appendix section B, Table table 2.

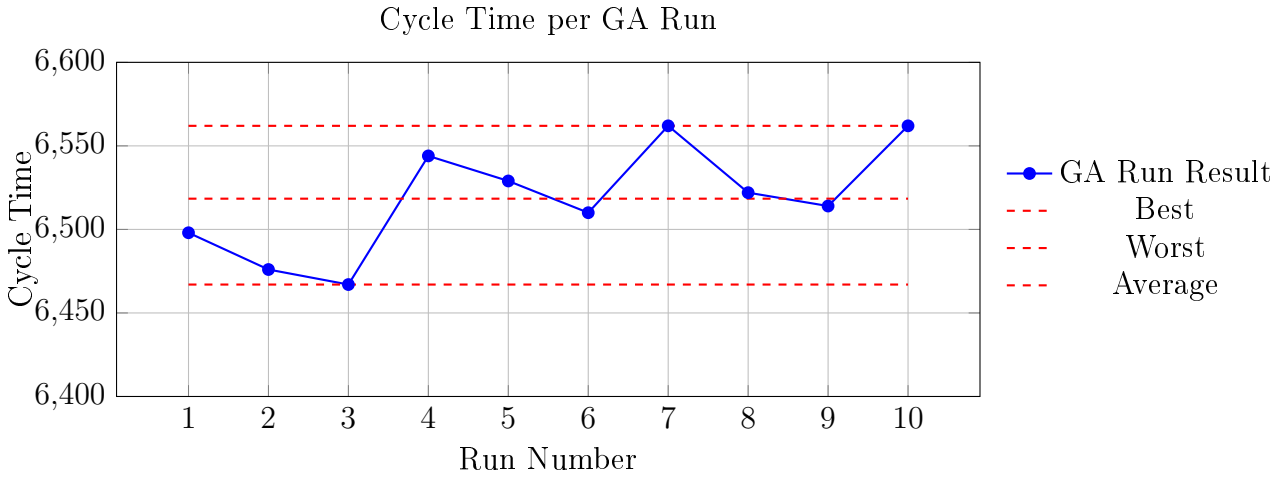

Figure 7: Instance ARC83 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

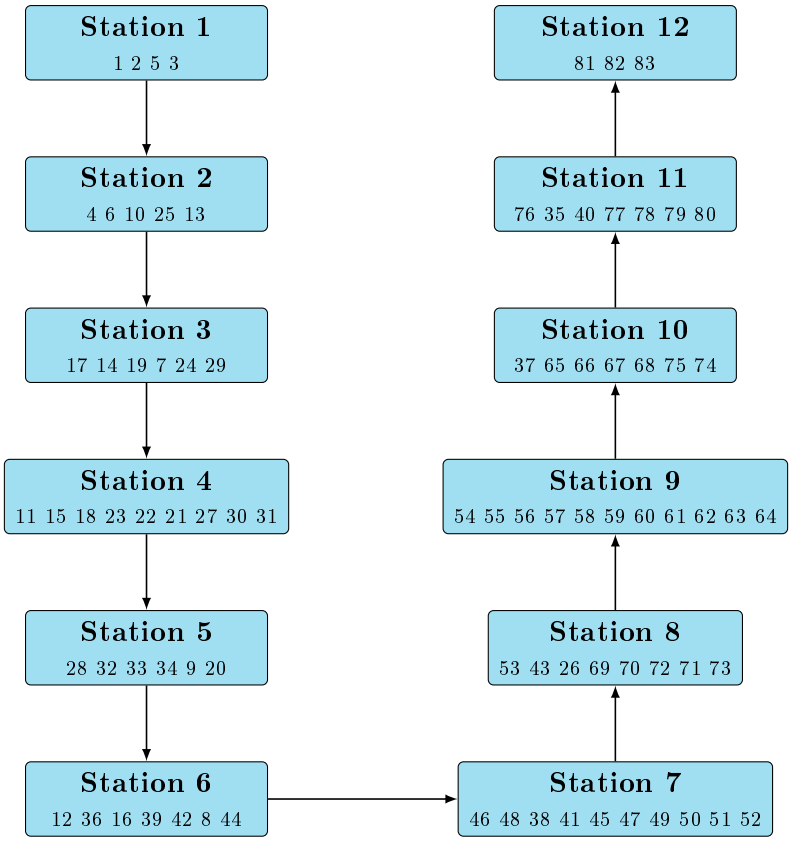

Figure 8: U-shaped 12-station assignment for ARC83 (Individual 1, Fitness = 0.000155).

4.4 Final Population and Top-10 Individuals

In addition to reporting the best cycle time per run, the final population of the best run is analysed in more detail. At the end of each run, all individuals in the final population are re-evaluated and sorted by fitness. For the best run, the Top-10 individuals are extracted and decoded.

Here, Individuals 1 and 2 correspond to two of the top 10 individuals for the ARC83 instance. The remaining individuals/solutions (Individuals 3–10) are reported in Appendix section C.

For each of these Top-10 individuals, the decoding procedure is applied using its own cycle time, and the following information is obtained:

- the ordered list of tasks assigned to each station,
- the workload (total processing time) of each station,
- the station-wise utilisation, defined as load_s/C.

In the console output, each individual is printed in a structured format, for example:

```
1) fitness=0.000271, cycle=3691
    Station 1: [1, 2, 3, 5, 8]
```

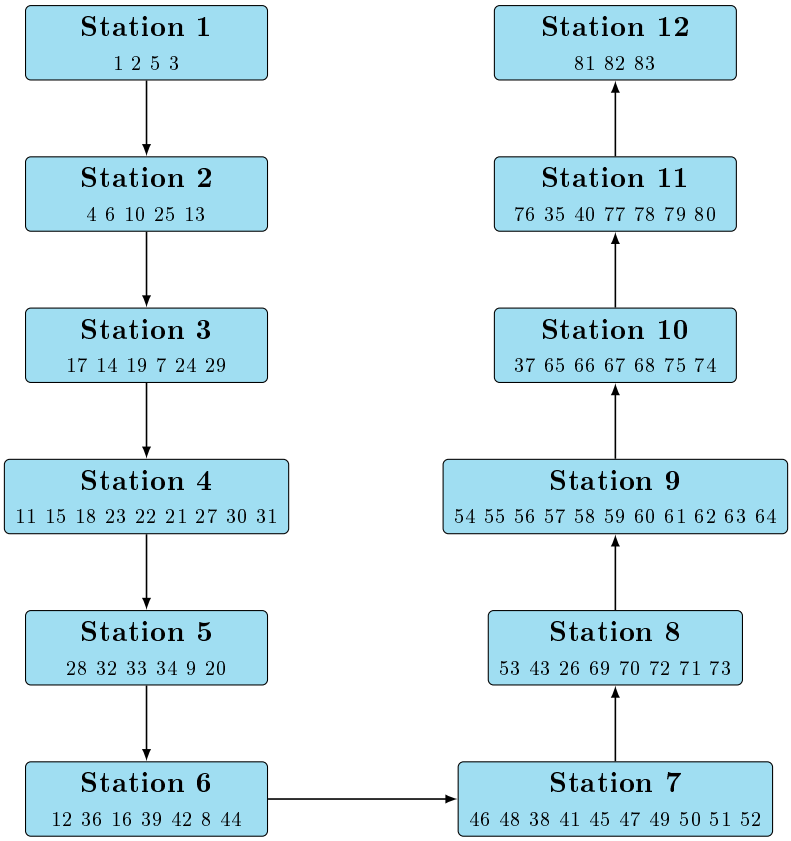

Figure 9: U-shaped 12-station assignment for ARC83 (Individual 2, Fitness = 0.000155).

```
Station 2: [4, 6, 7, 9]
Station 3: [10, 11, 12]
...
```

Furthermore, the implementation exports a CSV file named `<instance>-top-10-individuals-<timestamp>.csv`, in which each row corresponds to one station of one individual. The file contains the following fields:

- `Instance`: name of the precedence instance (e.g. ARC83),
- `Stations(m)`: number of stations m,
- `RankInBestRun`: rank of the individual within the Top-10 (1–10),
- `Fitness`: fitness value of the individual,
- `Cycle`: cycle time of the individual,
- `StationIndex`: index of the station (1,...,m),
- `StationLoad`: total processing time assigned to that station,
- `StationUtilization(Load/Cycle)`: station-wise utilisation,

- `TasksSequence`: sequence of tasks assigned to that station.

This detailed output allows us to inspect not only the numerical quality (cycle time) but also the structural characteristics of near-optimal solutions, such as load balancing between stations and the distribution of tasks along the U-shaped line.

4.5 Best Solution Analysis

GA_best_solution_details.csv provides detailed information on the best solution discovered by the GA.

4.5.1 Cycle Time and Line Efficiency

Best cycle time found by GA: 6467
The high efficiency indicates that task assignments utilize station capacities effectively within the limits imposed by the precedence structure.
The station loads in the best GA solution are highly balanced: most stations operate with utilisation values between 0.96 and 1.00, with only the last station (Station 12) dropping to 0.84 due to precedence constraints and the position of late tasks (81 - 83). The overall line efficiency of 97.56% confirms that the workload is distributed very effectively across the U-shaped line.
Figure **??** illustrates the U-shaped layout of the best GA solution for ARC83, showing station loads, utilisation levels and the distribution of tasks along the forward and return flows.

4.5.2 Station Load Distribution

The station loads in the best solution reveal the following pattern:

- Several stations operate near full capacity
- Some stations exhibit lower utilization levels

This is expected, as the ARC83 precedence network contains:

- Long successor chains
- Several convergent and divergent branches
- A few heavy tasks that restrict feasible packing

Typical for U-shaped lines:

- Early stations contain short tasks with dense predecessor requirements

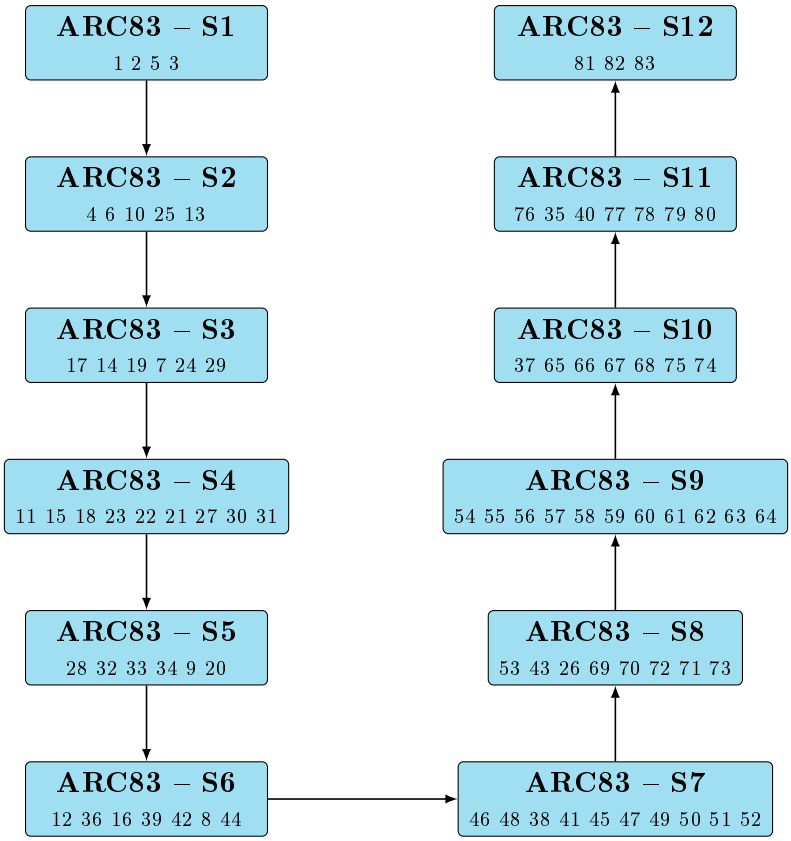

Figure 10: U-shaped 12-station assignment for ARC83 (BestCycle = 6431).

- Middle stations group flexible tasks

Overall, the load distribution closely matches patterns observed in high-quality UALBP-2 solutions in the literature[1].

4.5.3 Task Sequence Characteristics

The task sequences for the best assignment show the successful operation of precedence-preserving mechanisms:

- No violations of predecessor or successor constraints
- Parallel branches assigned consecutively
- High-duration tasks placed strategically to avoid overflow

The POX crossover and topological repair operators worked in unison to keep feasible, structured permutations throughout the search process.

4.6 Interpretation of Overall Results

Table 1: GA Statistics for Solved Benchmark Instances

Instance	Stations (m)	Best Cycle Time	Average Cycle Time	Worst Cycle Time	Avg. Runtime (s)	Number of Tasks (n)
ARC83	12	6431	6464.70	6480	138.359	83
ARC111	12	12538	12541.00	12545	91.062	111
GUNTHER	12	44	44.00	44	10.356	35
BARTHOLD	12	470	470.90	471	114.729	148
BARTHOL2	30	143	143.10	144	163.445	148
SCHOLL	25	2799	2800.70	2804	697.079	279
LUTZ1	12	1400	1400.00	1400	15.487	32
LUTZ3	12	138	138.00	138	36.887	89
SAWYER30	12	28	28.00	28	19.419	30
TONGE70	12	294	294.20	295	79.287	70
HAHN	10	1775	1775.00	1775	22.974	53

Although the problem complexity cannot be characterized solely by a single metric, the number of tasks (n) provides a reasonable first-order measure of instance size. In addition, the presence of precedence constraints significantly increases the difficulty of the problem by restricting the set of feasible task permutations. Table table 1 summarizes the overall performance of the proposed Genetic Algorithm (GA) on several widely used UALBP benchmark instances. The results obtained clearly demonstrate that the algorithm can produce high-quality and stable solutions under different problem sizes and structural characteristics.

A key observation is that the difference between the best, average, and worst run times is quite small for almost all examples. For some problems, such as GUNTHER, BARTHOL2, LUTZ1, LUTZ3, and SAWYER30, the GA converged to the same solution in each independent run, resulting in identical best, average, and worst values. This demonstrates that the algorithm exhibits a high level of robustness and repeatability, and that the proposed representation, operators, and parameter settings effectively guide the search towards strong local optima, avoiding excessive randomness.

In larger and more complex examples such as ARC83, ARC111, SCHOLL, and TONGE70, slightly wider but still limited variability was observed between runs. In these cases, the average cycle times remained quite close to the best values, confirming that the algorithm delivers stable performance even in more challenging priority structures. The fact that the worst-case results do not deviate significantly from the average supports the reliability of the GA despite its stochastic nature.

In terms of computational cost, the reported average processing times remain at reasonable levels for all samples. While small-scale problems can be solved within a few seconds, larger and more complex examples naturally require more computation time. However, even in the most challenging cases, the processing times obtained remain within practical limits for offline assembly line design and analysis. This indicates a positive balance between solution quality and computational cost.

Furthermore, the obtained line efficiency values indicate that the stations operate at largely high occupancy rates, pointing to an effective workload distribution with limited

idle time. Thus, the solutions are not only competitive in terms of cycle time but also offer operationally acceptable configurations in terms of assembly line balancing. All these findings confirm that the selected parameter configuration and priority-preserving genetic operators provide a reliable and effective framework for solving UALBP-2 problems.

5 Contribution to Sustainable Development Goals

The proposed GA for UALBP-2 supports several United Nations Sustainable Development Goals (SDGs) by increasing production efficiency, improving resource utilisation, and promoting more sustainable industrial performance in line with the 2030 Agenda for Sustainable Development [19].

SDG 8: Decent Work and Economic Growth

Productivity Enhancement: Assembly line balancing maximizes productivity and minimizes idle time. Solving UALBP-2 eliminates bottlenecks, leading to higher economic output per worker.
Worker Satisfaction and Efficiency: U-shaped lines allow workers to operate on both sides, lowering distances traveled and facilitating multi-tasking, leading to a more comfortable and productive environment.

SDG 9: Industry, Innovation, and Infrastructure

Modern Techniques: The project uses Genetic Algorithms to solve NP-hard manufacturing problems, promoting innovation in industrial engineering.
Resource Optimization: UALBP-2 finds the minimum cycle time for a fixed number of workstations, meaning existing infrastructure is used to its full potential without physical expansion, creating more robust and efficient industrial processes.

SDG 12: Responsible Consumption and Production

Waste Minimizing: In manufacturing, "idle time" is waste. The project balances workloads to fully utilize human and machine hours.
Flexibility: U-lines allow for better adjustment to changing demand. Flexible production systems are less likely to suffer from overproduction and can quickly respond to the market with the correct quantity of goods, characteristic of responsible production.

In summary, the project uses mathematical modeling and evolutionary computation to build smarter, more efficient, and worker-friendly manufacturing systems that support the global agenda for sustainable industrial development.

6 Conclusions

References

[1] C. Becker and A. Scholl, "A survey on problems and methods in generalized assembly line balancing," *European Journal of Operational Research*, vol. 168, no. 3, pp. 694–715, 2006.

[2] J. Miltenburg, "U-shaped production lines: A review of theory and practice," *International Journal of Production Economics*, vol. 70, no. 3, pp. 201–214, 2001.

[3] N. Boysen, M. Fliedner, and A. Scholl, "A classification of assembly line balancing problems," *European Journal of Operational Research*, vol. 183, no. 2, pp. 674–693, 2007.

[4] A. Scholl and C. Becker, "State-of-the-art exact and heuristic solution procedures for simple assembly line balancing," *European Journal of Operational Research*, vol. 168, no. 3, pp. 666–693, 2006.

[5] F. B. Talbot, J. H. Patterson, and W. V. Gehrlein, "A comparative evaluation of heuristic line balancing techniques," *Management Science*, vol. 32, no. 4, pp. 430–454, 1986.

[6] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: W. H. Freeman, 1979.

[7] T. L. Urban, "Note. optimal balancing of u-shaped assembly lines," *Management Science*, vol. 44, no. 5, pp. 738–741, 1998.

[8] A. Scholl and R. Klein, "Ulino: Optimally balancing u-shaped jit assembly lines," *International Journal of Production Research*, vol. 37, no. 4, pp. 721–736, 1999.

[9] I. Baybars, "A survey of exact algorithms for the simple assembly line balancing problem," *Management Science*, vol. 32, no. 8, pp. 909–932, 1986.

[10] G. R. Aase and N. C. Suresh, "A mathematical programming formulation for u-shaped assembly line balancing," *International Journal of Production Research*, vol. 41, no. 11, pp. 2545–2569, 2003.

[11] S. Ghosh and C. Gagné, "A comprehensive review of assembly line balancing," *International Journal of Production Research*, 2011.

[12] Y. Kara, A. İşlier, and Y. Atasagun, "Balancing u-shaped assembly lines: heuristic and metaheuristic approaches," *Applied Mathematical Modelling*, 2010.

[13] E. Erel and S. Sarin, "A survey of the assembly line balancing problem," *Production Planning & Control*, 1998.

[14] T. Blickle and L. Thiele, "A comparison of selection schemes used in genetic algorithms," in *EVO Conference Proceedings*, 1996.

[15] Y. Park, Y. Kim, and S. Lee, "Precedence preserving genetic operators for scheduling," *Computers & Operations Research*, 2003.

[16] C. Reeves, "Genetic algorithms for the operations researcher," INFORMS, 1993.

[17] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning.* Reading, MA: Addison-Wesley, 1989.

[18] C. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity.* Dover, 1998.

[19] United Nations General Assembly, "Transforming our world: the 2030 agenda for sustainable development," Resolution A/RES/70/1, 2015, adopted 25 September 2015.

A Genetic Algorithm Code for ARC83 (UALBP-II)

Listing 1: Python implementation of the genetic algorithm for the U-shaped assembly line balancing problem (UALBP-II) on the ARC83 instance.

```
import random
import math
import time
from typing import List, Tuple, Dict, Set
import csv
import os
from datetime import datetime

# ============================================================
# KULLANICI AYARLARI
# ============================================================
INSTANCE_PATH      = "ARC83.IN2"    # IN2 dosyanin yolu
M_STATIONS         = 12           # istasyon sayisi
POP_SIZE           = 40
GENERATIONS        = 300 #gorsellestirme
CROSSOVER_RATE     = 0.7
MUTATION_RATE      = 0.08
RUNS               = 10          # GA kac kez calisacak
BASE_SEED          = 0

OPTIMAL_CYCLE      = 6467        # Excel'den bildigimiz optimum AMA
    SALBP ICIN

# ============================================================
# POX DEBUG AYARLARI
# ============================================================
DEBUG_POX          = False          # POX ornegi gormek istemezsen
    False yap
POX_DEBUG_LIMIT    = 5             # En fazla kac crossover ornegi
    yazdirilsin
pox_debug_counter = 0


# ============================================================
# 1) IN2 DOSYASINI OKUMA
# ============================================================

```

```
35 def read_in2(path: str) -> Tuple[int, List[int], List[Tuple[int,
       int]]]:
36     with open(path, "r") as f:
37         lines = [line.strip() for line in f if line.strip()]
38 
39     n = int(lines[0])
40 
41     times = []
42     for i in range(1, n + 1):
43         times.append(int(lines[i]))
44 
45     precedences = []
46     for line in lines[n + 1:]:
47         if "," in line:
48             a, b = line.split(",")
49         else:
50             a, b = line.split()
51         i = int(a)
52         j = int(b)
53         if i == -1 and j == -1:
54             break
55         precedences.append((i, j))
56 
57     return n, times, precedences
58 
59 
60 # ============================================================
61 # 2) ONCUL / ARDIL KUMELERI
62 # ============================================================
63 
64 def build_pred_succ(
65     n: int,
66     precedences: List[Tuple[int, int]]
67 ) -> Tuple[Dict[int, Set[int]], Dict[int, Set[int]]]:
68     preds = {i: set() for i in range(1, n + 1)}
69     succs = {i: set() for i in range(1, n + 1)}
70 
71     for i, j in precedences:
72         preds[j].add(i)
73         succs[i].add(j)
74 
```

```
    return preds, succs


# ============================================================
# 3) TOPOLOJIK SIRALAMA + REPAIR
# ============================================================

def random_topological_sort(
    n: int,
    preds: Dict[int, Set[int]],
    rng: random.Random
) -> List[int]:
    remaining = set(range(1, n + 1))
    result = []
    assigned = set()

    while remaining:
        eligible = [i for i in remaining if preds[i].issubset(
           assigned)]
        if not eligible:
            raise RuntimeError("Precedence graph contains a cycle?"
               )
        chosen = rng.choice(eligible)
        result.append(chosen)
        assigned.add(chosen)
        remaining.remove(chosen)

    return result


def repair_to_topological(
    perm: List[int],
    preds: Dict[int, Set[int]]
) -> List[int]:
    rank = {task: idx for idx, task in enumerate(perm)}
    remaining = set(perm)
    result = []
    placed = set()

    while remaining:
```

```
        eligible = [i for i in remaining if preds[i].issubset(
            placed)]
        if not eligible:
            eligible = list(remaining)
        chosen = min(eligible, key=lambda x: rank[x])
        result.append(chosen)
        placed.add(chosen)
        remaining.remove(chosen)

    return result


# ============================================================
# 4) U-SHAPED DECODE + CYCLE TIME
# ============================================================

def is_task_eligible(
    task: int,
    assigned: Set[int],
    preds: Dict[int, Set[int]],
    succs: Dict[int, Set[int]]
) -> bool:
    return preds[task].issubset(assigned) or succs[task].issubset(
        assigned)


def decode_with_cycle_limit(
    perm: List[int],
    times: List[int],
    preds: Dict[int, Set[int]],
    succs: Dict[int, Set[int]],
    m: int,
    c: int
) -> Tuple[bool, List[List[int]], List[int]]:
    n = len(perm)
    assigned: Set[int] = set()
    stations: List[List[int]] = [[] for _ in range(m)]
    loads: List[int] = [0 for _ in range(m)]

    station_idx = 0

```

```
    while station_idx < m and len(assigned) < n:
        progress = True
        while progress and len(assigned) < n:
            progress = False
            for task in perm:
                if task in assigned:
                    continue
                if not is_task_eligible(task, assigned, preds,
                   succs):
                    continue

                duration = times[task - 1]
                if loads[station_idx] + duration <= c:
                    stations[station_idx].append(task)
                    loads[station_idx] += duration
                    assigned.add(task)
                    progress = True
        station_idx += 1

    feasible = (len(assigned) == n)
    return feasible, stations, loads


def evaluate_permutation_cycle_time(
    perm: List[int],
    times: List[int],
    preds: Dict[int, Set[int]],
    succs: Dict[int, Set[int]],
    m: int
) -> Tuple[float, List[List[int]], List[int]]:
    total_time = sum(times)
    max_time = max(times)

    lb = max(max_time, math.ceil(total_time / m))
    ub = total_time

    best_c = None
    best_stations = None
    best_loads = None

    while lb <= ub:
```

```
            mid = (lb + ub) // 2
            feasible, stations, loads = decode_with_cycle_limit(
                perm, times, preds, succs, m, mid
            )
            if feasible:
                best_c = mid
                best_stations = stations
                best_loads = loads
                ub = mid - 1
            else:
                lb = mid + 1

    if best_c is None:
        return float("inf"), [[] for _ in range(m)], [0 for _ in
            range(m)]

    return float(best_c), best_stations, best_loads


# ============================================================
# 5) GA OPERATORLERI
# ============================================================

def tournament_selection(
    population: List[List[int]],
    fitness: List[float],
    rng: random.Random,
    k: int = 2
) -> List[int]:
    n = len(population)
    best_idx = None
    best_fit = -float("inf")
    for _ in range(k):
        i = rng.randrange(n)
        if fitness[i] > best_fit:
            best_fit = fitness[i]
            best_idx = i
    return population[best_idx][:]


def pox_crossover(
```

```
    p1: List[int],
    p2: List[int],
    rng: random.Random
) -> List[int]:
    """
    POX crossover + istege bagli debug ciktilari.
    """
    global pox_debug_counter

    chosen = set()
    for gene in p1:
        if rng.random() < 0.5:
            chosen.add(gene)
    if not chosen:
        chosen.add(rng.choice(p1))

    child = []
    for gene in p1:
        if gene in chosen:
            child.append(gene)
    for gene in p2:
        if gene not in chosen:
            child.append(gene)

    # ---- Debug output ----
    if DEBUG_POX and pox_debug_counter < POX_DEBUG_LIMIT:
        pox_debug_counter += 1
        print("\n=== POX CROSSOVER SAMPLE ===")
        print("Parent1 (first 15):", p1[:15])
        print("Parent2 (first 15):", p2[:15])
        print("Chosen subset:", sorted(chosen))
        print("Child   (first 15):", child[:15])
        print("============================\n")

    return child


def swap_mutation(perm: List[int], rng: random.Random) -> List[int
    ]:
    n = len(perm)
    i, j = rng.sample(range(n), 2)
```

```
    perm[i], perm[j] = perm[j], perm[i]
    return perm


# ============================================================
# 6) GA: TEK RUN + ISTATISTIK
# ============================================================

def genetic_algorithm_ualbp2(
    n: int,
    times: List[int],
    preds: Dict[int, Set[int]],
    succs: Dict[int, Set[int]],
    m: int,
    pop_size: int,
    generations: int,
    crossover_rate: float,
    mutation_rate: float,
    seed: int,
    run_index: int
):
    rng = random.Random(seed)

    population: List[List[int]] = []
    for _ in range(pop_size):
        perm = random_topological_sort(n, preds, rng)
        population.append(perm)

    best_perm = None
    best_cycle = float("inf")
    best_stations = None
    best_loads = None

    # ----- CONVERGENCE KAYDI (jenerasyon bazli) -----
    history_best_so_far = []
    history_best_gen = []
    history_mean_gen = []

    for gen in range(generations):
        fitness_vals = []
        eval_cache = {}
```

```
313
314         for perm in population:
315             key = tuple(perm)
316             if key in eval_cache:
317                 cycle_time = eval_cache[key]
318             else:
319                 cycle_time, stations, loads =
                        evaluate_permutation_cycle_time(
320                     perm, times, preds, succs, m
321                 )
322                 eval_cache[key] = cycle_time
323                 if cycle_time < best_cycle:
324                     best_cycle = cycle_time
325                     best_perm = perm[:]
326                     best_stations = stations
327                     best_loads = loads
328
329             if math.isinf(eval_cache[key]):
330                 fit = 0.0
331             else:
332                 fit = 1.0 / eval_cache[key]
333             fitness_vals.append(fit)
334
335         gen_cycles = list(eval_cache.values())
336         gen_best = min(gen_cycles)
337         gen_mean = sum(gen_cycles) / len(gen_cycles)
338
339         history_best_gen.append(gen_best)
340         history_mean_gen.append(gen_mean)
341         history_best_so_far.append(best_cycle)
342
343         new_population: List[List[int]] = []
344         if best_perm is not None:
345             new_population.append(best_perm[:])
346
347         while len(new_population) < pop_size:
348             p1 = tournament_selection(population, fitness_vals, rng
                    , k=2)
349             p2 = tournament_selection(population, fitness_vals, rng
                    , k=2)
350
```

```
            if rng.random() < crossover_rate:
                child = pox_crossover(p1, p2, rng)
            else:
                child = p1[:]

            if rng.random() < mutation_rate:
                child = swap_mutation(child, rng)

            child = repair_to_topological(child, preds)
            new_population.append(child)

        population = new_population

    # ----- RUN SONU: final populasyon istatistikleri -----
    final_stats = []
    cycles = []
    fits = []

    for perm in population:
        cycle_time, _, _ = evaluate_permutation_cycle_time(
            perm, times, preds, succs, m
        )
        fit = 0.0 if math.isinf(cycle_time) else 1.0 / cycle_time
        cycles.append(cycle_time)
        fits.append(fit)
        final_stats.append((fit, cycle_time, perm))

    best_cycle_final = min(cycles)
    worst_cycle_final = max(cycles)
    mean_cycle_final = sum(cycles) / len(cycles)

    best_fit_final = max(fits)
    worst_fit_final = min(fits)
    mean_fit_final = sum(fits) / len(fits)

    final_stats.sort(key=lambda x: x[0], reverse=True)
    top10 = final_stats[:10]

    run_stats = {
        "best_cycle": best_cycle,
        "final_best_cycle": best_cycle_final,
```

```
        "final_mean_cycle": mean_cycle_final,
        "final_worst_cycle": worst_cycle_final,
        "final_best_fit": best_fit_final,
        "final_mean_fit": mean_fit_final,
        "final_worst_fit": worst_fit_final,
        "top10": top10,
        "history_best_so_far": history_best_so_far,
        "history_best_gen": history_best_gen,
        "history_mean_gen": history_mean_gen,
    }

    return best_perm, best_cycle, best_stations, best_loads,
       run_stats


# ============================================================
# 7) COKLU RUN + OZET
# ============================================================

def main():
    start_time = time.time()

    n, times, precedences = read_in2(INSTANCE_PATH)
    preds, succs = build_pred_succ(n, precedences)

    total_time = sum(times)
    max_time = max(times)
    lb1 = math.ceil(total_time / M_STATIONS)
    lb2 = max_time
    lb = max(lb1, lb2)

    print(f"Instance: {INSTANCE_PATH}")
    print(f"Tasks (n)              : {n}")
    print(f"Stations (m)           : {M_STATIONS}")
    print(f"Total work content sum : {total_time}")
    print(f"Max task time          : {max_time}")
    print(f"Lower bound LB1=ceil(sum/m) = {lb1}")
    print(f"Lower bound LB2=max(t_i)    = {lb2}")
    print(f"Overall theoretical LB       = {lb}")
    print()

```

```
    all_convergence_rows = []
    run_best_cycles = []
    run_runtimes = []
    best_overall_cycle = float("inf")
    best_overall_solution = None
    best_overall_top10 = None

    for r in range(1, RUNS + 1):
        seed = BASE_SEED + r
        print(f"\n============================")
        print(f"         RUN {r}/{RUNS}")
        print(f"============================")
        run_start = time.time()

        best_perm, best_cycle, best_stations, best_loads, stats =
           genetic_algorithm_ualbp2(
            n=n,
            times=times,
            preds=preds,
            succs=succs,
            m=M_STATIONS,
            pop_size=POP_SIZE,
            generations=GENERATIONS,
            crossover_rate=CROSSOVER_RATE,
            mutation_rate=MUTATION_RATE,
            seed=seed,
            run_index=r
        )

        hb  = stats["history_best_so_far"]
        hbg = stats["history_best_gen"]
        hm  = stats["history_mean_gen"]

        for g in range(GENERATIONS):
            all_convergence_rows.append([r, g + 1, hb[g], hbg[g],
               hm[g]])

        run_end = time.time()
        run_runtime = run_end - run_start

        run_best_cycles.append(best_cycle)
```

```
        run_runtimes.append(run_runtime)

        print(f"\nRun {r} summary:")
        print(f"  Best cycle time over generations      : {best_cycle}")
        print(f"  Final population best / mean / worst c: "
              f"{stats['final_best_cycle']} / {stats['final_mean_cycle']:.2f} / {stats['final_worst_cycle']}")
        print(f"  Final population best / mean / worst fitness: "
              f"{stats['final_best_fit']:.6f} / {stats['final_mean_fit']:.6f} / {stats['final_worst_fit']:.6f}")

        print("\n  Top 10 individuals in final population:")
        for rank, (fit, cyc, perm) in enumerate(stats["top10"], start=1):
            print(f"    {rank:2d}) fitness={fit:.6f}, cycle={cyc}")
            feasible, stations, loads = decode_with_cycle_limit(
                perm, times, preds, succs, M_STATIONS, int(cyc)
            )
            for s_idx, tasks in enumerate(stations, start=1):
                print(f"        Station {s_idx}: {tasks}")

        if best_cycle < best_overall_cycle:
            best_overall_cycle = best_cycle
            best_overall_solution = (best_perm, best_stations, best_loads)
            best_overall_top10 = stats["top10"]

    mean_cycle_runs = sum(run_best_cycles) / len(run_best_cycles)
    worst_cycle_runs = max(run_best_cycles)

    best_gap_abs = best_overall_cycle - OPTIMAL_CYCLE
    best_gap_pct = 100.0 * best_gap_abs / OPTIMAL_CYCLE

    mean_gap_abs = mean_cycle_runs - OPTIMAL_CYCLE
    mean_gap_pct = 100.0 * mean_gap_abs / OPTIMAL_CYCLE

    total_runtime = time.time() - start_time

```

```
505     print("\n====================================")
506     print("         10 RUN GLOBAL SUMMARY")
507     print("====================================")
508     print(f"GA parameters:")
509     print(f"  Population size    : {POP_SIZE}")
510     print(f"  Generations        : {GENERATIONS}")
511     print(f"  Crossover rate     : {CROSSOVER_RATE}")
512     print(f"  Mutation rate      : {MUTATION_RATE}")
513     print(f"  Parent selection   : Tournament (k=2)")
514     print(f"  Crossover method   : POX-benzeri precedence preserving
        ")
515     print(f"  Mutation method    : Swap mutation")
516     print(f"  Iteration number   : {GENERATIONS} generations per run
        ")
517     print(f"  Total runs         : {RUNS}")
518     print()
519
520     print(f"Run best cycle times: {run_best_cycles}")
521     print(f"Best  cycle over runs : {best_overall_cycle}")
522     print(f"Mean  cycle over runs : {mean_cycle_runs:.2f}")
523     print(f"Worst cycle over runs : {worst_cycle_runs}")
524     print()
525
526     print(f"Known optimal C*       : {OPTIMAL_CYCLE}")
527     print(f"Best-of-10 gap         : {best_gap_abs}  ({best_gap_pct
        :.3f} % above optimum)")
528     print(f"Mean-of-10 gap         : {mean_gap_abs:.2f}  ({
        mean_gap_pct:.3f} % above optimum)")
529     print()
530
531     best_perm, best_stations, best_loads = best_overall_solution
532     print("Best-of-10 solution station loads:")
533     for i, load in enumerate(best_loads, start=1):
534         print(f"  Station {i}: load = {load}")
535
536     print("\nBest-of-10 solution assignments:")
537     for i, tasks in enumerate(best_stations, start=1):
538         print(f"  Station {i}: {tasks}")
539
540     print("\nTop 10 individuals of best run (by fitness):")
```

```
    for rank, (fit, cyc, perm) in enumerate(best_overall_top10, start=1):
        print(f"  {rank:2d}) fitness={fit:.6f}, cycle={cyc}")
        feasible, stations, loads = decode_with_cycle_limit(
            perm, times, preds, succs, M_STATIONS, int(cyc)
        )
        for s_idx, tasks in enumerate(stations, start=1):
            print(f"        Station {s_idx}: {tasks}")

    print(f"\nTotal runtime for {RUNS} runs: {total_runtime:.2f} seconds")

    instance_name = os.path.splitext(os.path.basename(INSTANCE_PATH))[0]
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M")

    summary_file      = f"{instance_name}_summary_{timestamp}.csv"
    runs_file         = f"{instance_name}_run_details_{timestamp}.csv"
    best_details_file = f"{instance_name}_best_details_{timestamp}.csv"
    top10_file        = f"{instance_name}-top-10-individuals-{timestamp}.csv"
    convergence_file  = f"{instance_name}_convergence_{timestamp}.csv"

    print("\nGenerated output filenames:")
    print("  Summary file      :", summary_file)
    print("  Run details file  :", runs_file)
    print("  Best details file :", best_details_file)
    print("  Top10 file        :", top10_file)
    print()

    with open(summary_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow([
            "Instance","Stations(m)","TotalWork(sum_ti)","Population","Generations",
            "CrossoverRate","MutationRate","ParentSelection","CrossoverMethod",
```

```
572             "MutationMethod","Runs","BestCycle","MeanCycle","
                    WorstCycle","OptimalC",
573             "BestGapAbs","BestGapPct","MeanGapAbs","MeanGapPct","
                    TotalRuntime(sec)"
574         ])
575         writer.writerow([
576             instance_name, M_STATIONS, total_time, POP_SIZE,
                    GENERATIONS,
577             CROSSOVER_RATE, MUTATION_RATE, "Tournament(k=2)", "POX"
                    , "Swap", RUNS,
578             best_overall_cycle, f"{mean_cycle_runs:.2f}",
                    worst_cycle_runs, OPTIMAL_CYCLE,
579             best_gap_abs, f"{best_gap_pct:.3f}", f"{mean_gap_abs:.2
                    f}", f"{mean_gap_pct:.3f}",
580             f"{total_runtime:.2f}"
581         ])
582
583     print(f"CSV summary saved to: {summary_file}\n")
584
585     with open(runs_file, "w", newline="", encoding="utf-8") as f:
586         writer = csv.writer(f)
587         writer.writerow(["Run","BestCycle","RunRuntime(sec)"])
588         for r in range(len(run_best_cycles)):
589             writer.writerow([r + 1, run_best_cycles[r], f"{
                    run_runtimes[r]:.2f}"])
590
591     print(f"Run details saved to: {runs_file}")
592
593     total_line_eff = 100.0 * total_time / (M_STATIONS *
            best_overall_cycle)
594
595     with open(best_details_file, "w", newline="", encoding="utf-8")
            as f:
596         writer = csv.writer(f)
597         writer.writerow([
598             "Instance","Stations(m)","BestCycle","TotalWork(sum_ti)
                    ","GlobalLineEfficiency(%)",
599             "StationIndex","StationLoad","StationUtilization(Load/
                    BestCycle)","TasksSequence"
600         ])
```

```
601         for s_idx, (load, tasks) in enumerate(zip(best_loads,
                best_stations), start=1):
602             utilization = load / best_overall_cycle
603             tasks_str = " ".join(str(t) for t in tasks)
604             writer.writerow([
605                 instance_name, M_STATIONS, best_overall_cycle,
                        total_time,
606                 f"{total_line_eff:.2f}", s_idx, load, f"{
                        utilization:.4f}", tasks_str
607             ])
608
609     print(f"Best solution details saved to: {best_details_file}")
610
611     with open(top10_file, "w", newline="", encoding="utf-8") as f:
612         writer = csv.writer(f)
613         writer.writerow([
614             "Instance","Stations(m)","RankInBestRun","Fitness","
                    Cycle","StationIndex",
615             "StationLoad","StationUtilization(Load/Cycle)","
                    TasksSequence"
616         ])
617         for rank, (fit, cyc, perm) in enumerate(best_overall_top10,
                start=1):
618             feasible, stations, loads = decode_with_cycle_limit(
619                 perm, times, preds, succs, M_STATIONS, int(cyc)
620             )
621             for s_idx, (load, tasks) in enumerate(zip(loads,
                    stations), start=1):
622                 tasks_str = " ".join(str(t) for t in tasks)
623                 utilization = 0.0 if cyc == 0 else load / cyc
624                 writer.writerow([
625                     instance_name, M_STATIONS, rank, f"{fit:.6f}",
                            cyc,
626                     s_idx, load, f"{utilization:.4f}", tasks_str
627                 ])
628
629     print(f"Top 10 individuals (best run) saved to: {top10_file}")
630
631     with open(convergence_file, "w", newline="", encoding="utf-8")
            as f:
632         writer = csv.writer(f)
```

```
633         writer.writerow(["Run","Generation","BestSoFar","
               BestInGeneration","MeanCycle"])
634         writer.writerows(all_convergence_rows)
635
636     print(f"Convergence history saved to: {convergence_file}")
637
638
639 if __name__ == "__main__":
640     main()
```

Table 2: Run-by-run best cycle time and runtime results for all benchmark instances.

(a) BARTHOLD

Run	BestCycle	Runtime (s)
1	470	40.56
2	471	39.63
3	471	38.53
4	471	41.16
5	471	43.95
6	471	43.33
7	471	40.14
8	471	43.86
9	471	44.41
10	471	43.95

(b) TONGE70

Run	BestCycle	Runtime (s)
1	210	22.10
2	210	25.35
3	209	22.37
4	210	26.65
5	210	22.35
6	209	20.31
7	210	21.34
8	210	20.61
9	210	20.40
10	210	20.53

(c) SAWYER30

Run	BestCycle	Runtime (s)
1	47	3.00
2	47	3.37
3	47	3.15
4	47	3.29
5	47	3.22
6	47	3.32
7	47	3.54
8	47	3.24
9	47	3.69
10	47	3.61

(d) LUTZ1

Run	BestCycle	Runtime (s)
1	1400	7.93
2	1400	8.13
3	1400	7.89
4	1400	8.04
5	1400	8.05
6	1400	7.92
7	1400	7.83
8	1400	7.53
9	1400	7.64
10	1400	7.52

Table 2: Run-by-run best cycle time and runtime results for all benchmark instances (continued).

(e) ARC111

Run	BestCycle	Runtime (s)
1	12542	44.32
2	12546	43.13
3	12549	43.63
4	12547	43.07
5	12549	42.76
6	12546	43.46
7	12550	40.49
8	12549	42.95
9	12550	42.20
10	12543	42.57

(f) LUTZ3

Run	BestCycle	Runtime (s)
1	548	9.68
2	548	8.93
3	548	8.30
4	548	8.27
5	548	8.23
6	548	8.72
7	548	8.60
8	548	8.15
9	548	8.54
10	548	8.37

(g) SCHOLL

Run	BestCycle	Runtime (s)
1	1825	275.45
2	1827	290.46
3	1821	299.74
4	1824	285.49
5	1818	273.51
6	1829	291.93
7	1821	284.44
8	1827	285.68
9	1823	287.15
10	1823	334.99

(h) ARC83

Run	BestCycle	Runtime (s)
1	6467	25.89
2	6562	26.65
3	6488	25.94
4	6534	25.53
5	6562	25.87
6	6508	25.50
7	6512	26.62
8	6560	30.12
9	6544	26.70
10	6467	29.45

B Run-by-run Results for All Benchmark Instances

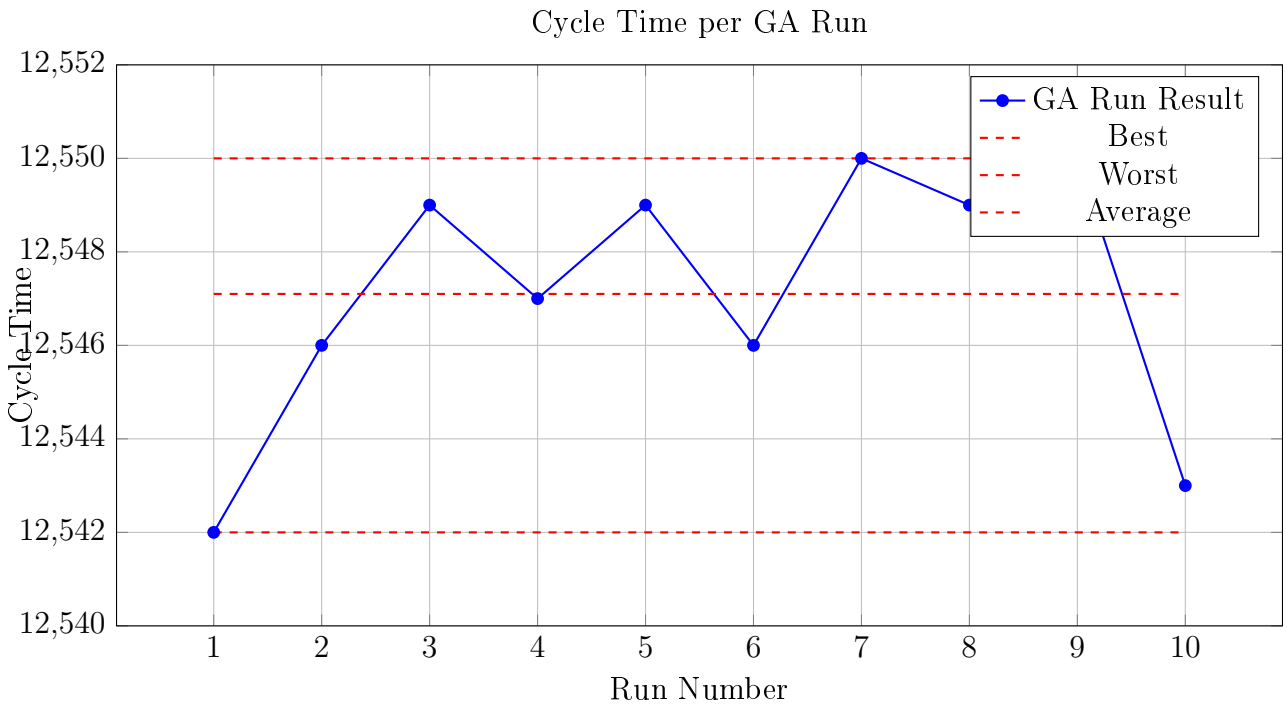

Figure 11: Instance ARC83, Cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

Table 2: Run-by-run best cycle time and runtime results for all benchmark instances (continued).

(i) BARTHOL2

Run	BestCycle	Runtime (s)
1	159	66.58
2	159	71.32
3	159	69.39
4	159	61.17
5	159	60.83
6	159	60.94
7	159	61.24
8	159	62.09
9	159	63.06
10	159	61.65

(j) GUNTHER

Run	BestCycle	Runtime (s)
1	44	5.09
2	44	5.46
3	44	6.34
4	44	6.25
5	44	5.30
6	44	5.04
7	44	5.49
8	44	6.48
9	44	5.58
10	44	5.44

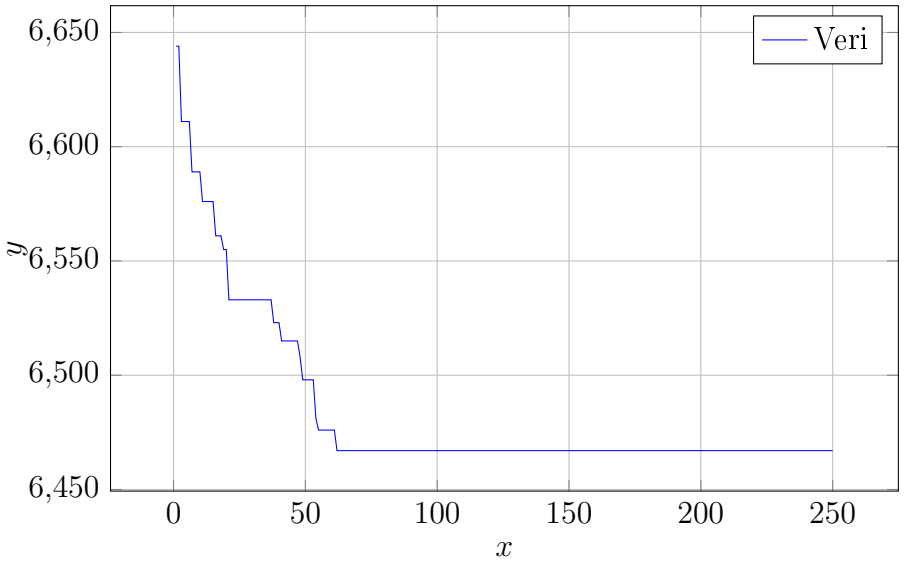

Figure 12: CSV dosyasından çizilen grafik

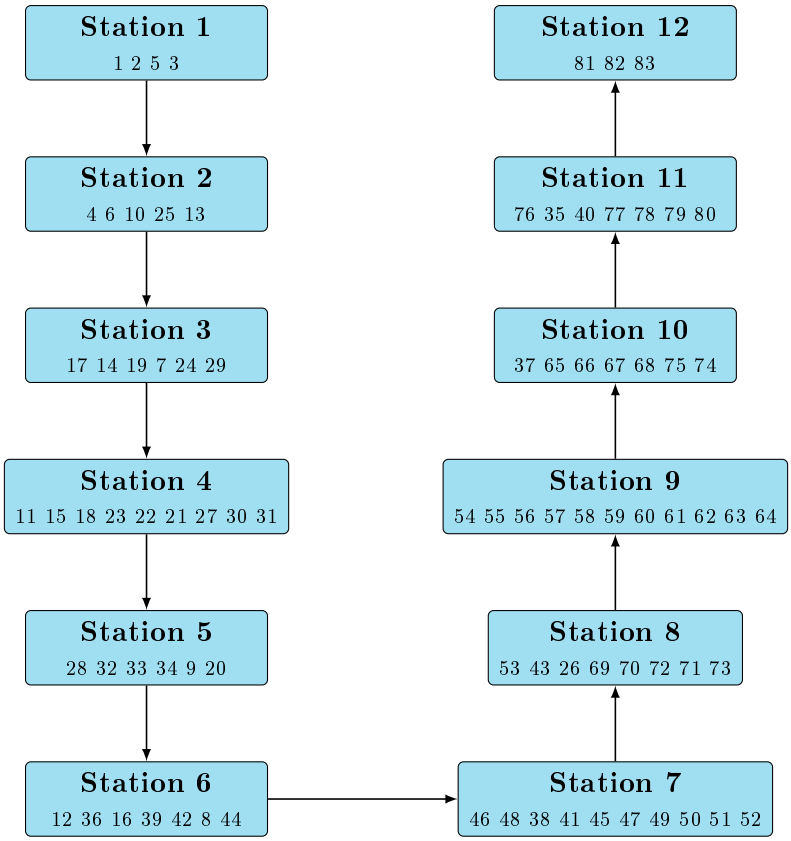

Figure 13: U-shaped 12-station assignment for ARC83 (Individual 3, Fitness = 0.000155).

C Additional Results for ARC83' top 10 Individuals

D Line Graphs For 9 Instances

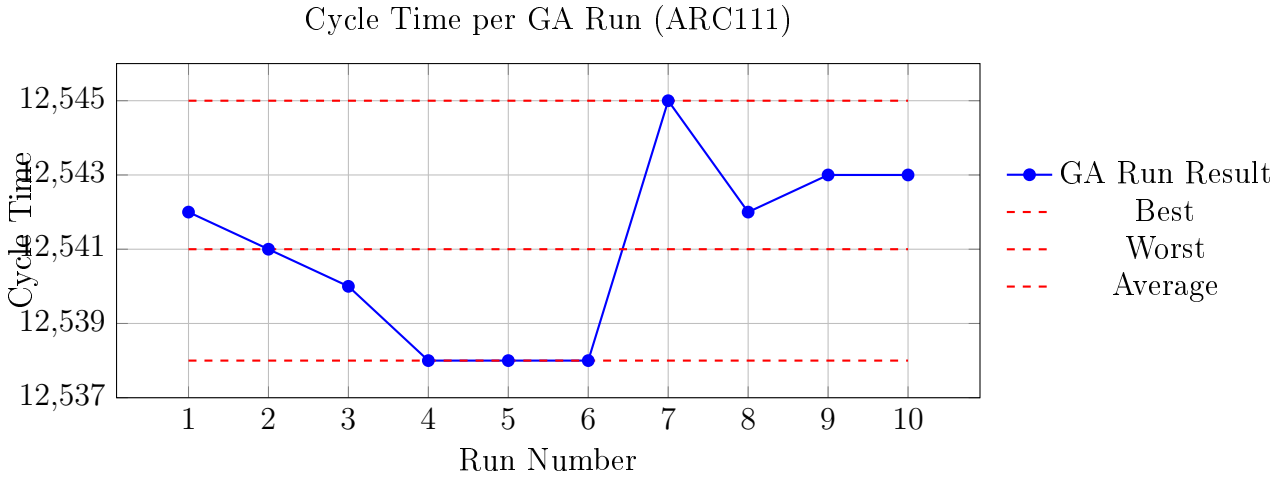

Figure 21: Instance ARC111 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

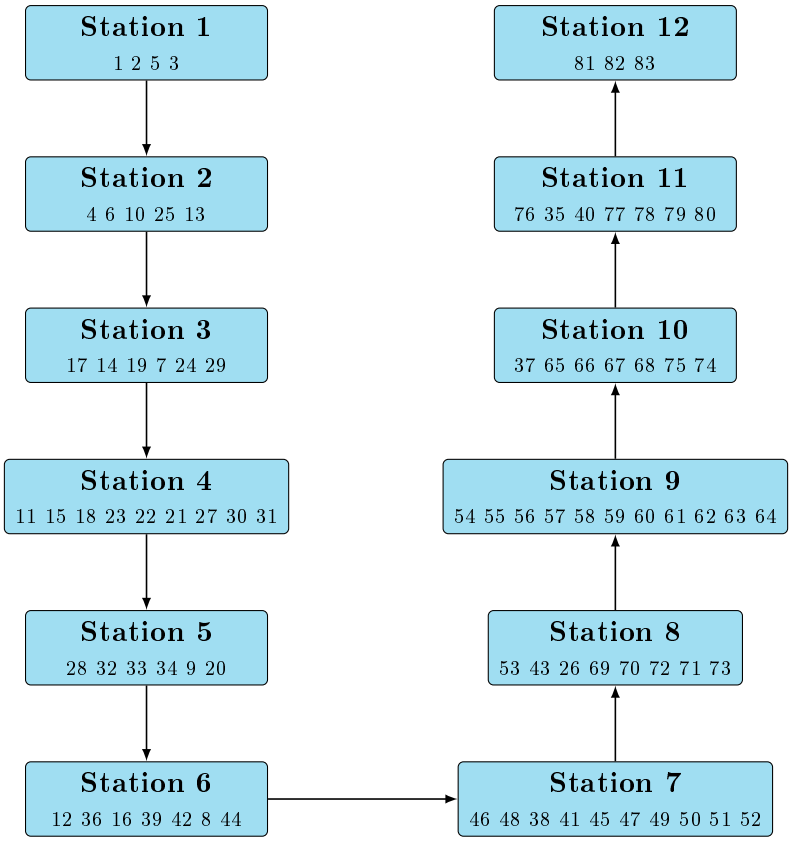

Figure 14: U-shaped 12-station assignment for ARC83 (Individual 4, Fitness = 0.000155).

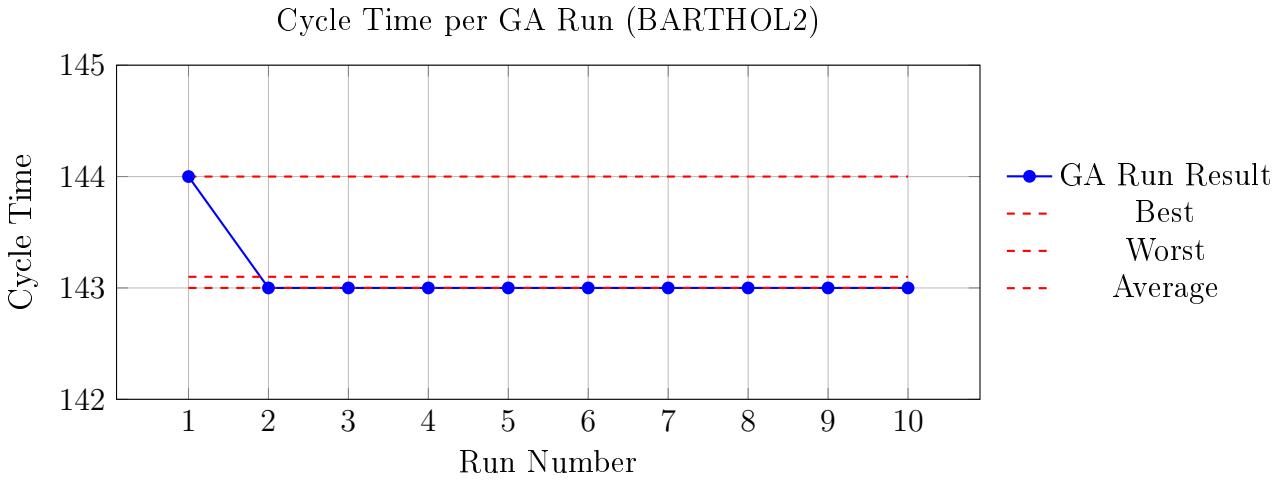

Figure 22: Instance BARTHOL2 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

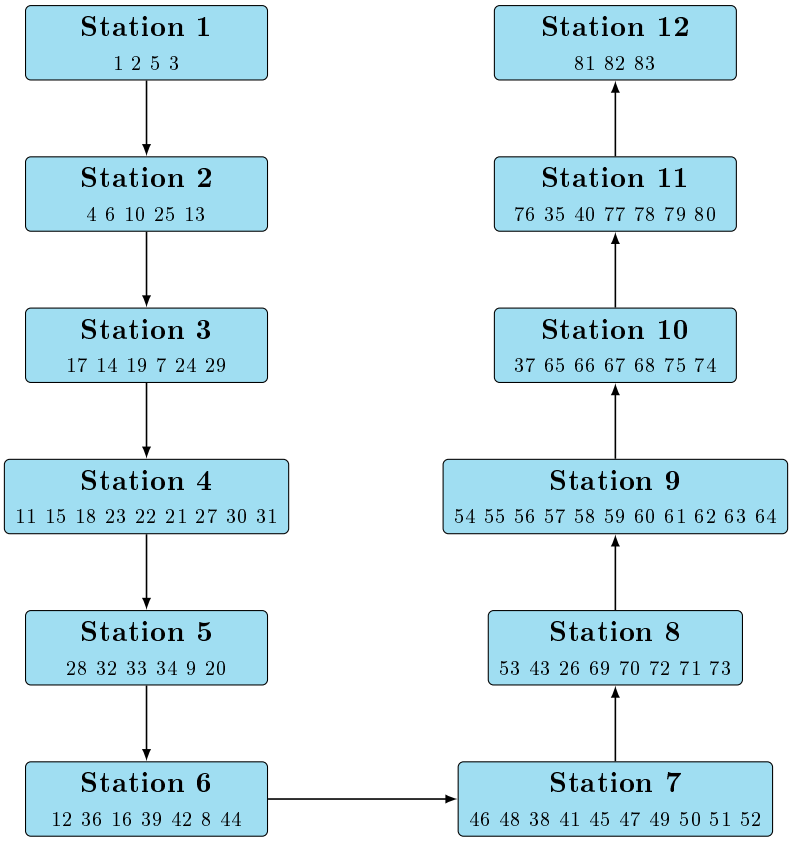

Figure 15: U-shaped 12-station assignment for ARC83 (Individual 5, Fitness = 0.000155).

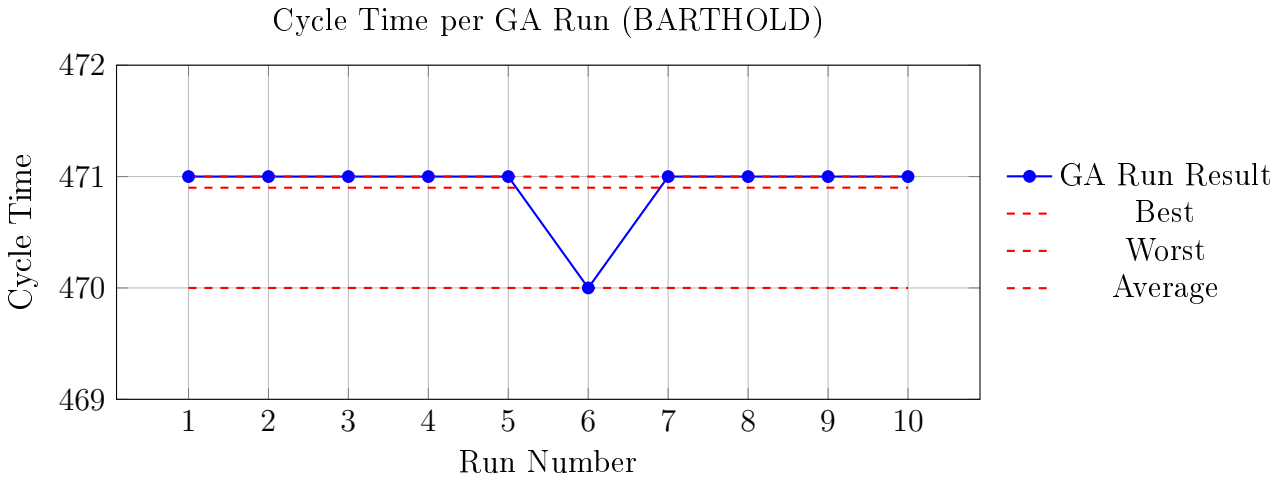

Figure 23: Instance BARTHOLD cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

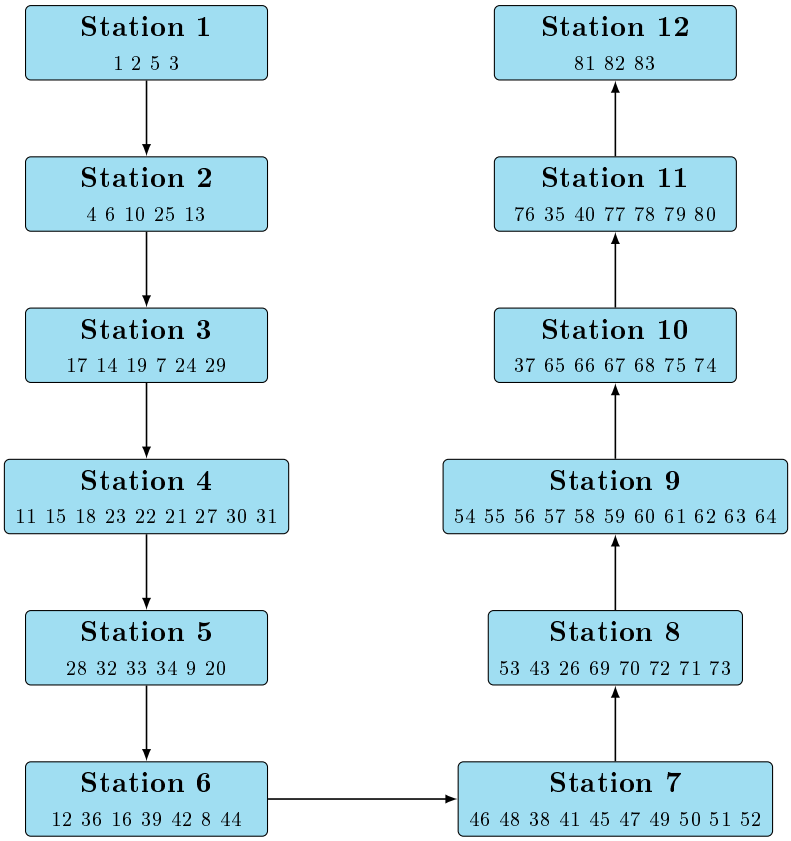

Figure 16: U-shaped 12-station assignment for ARC83 (Individual 6, Fitness = 0.000155).

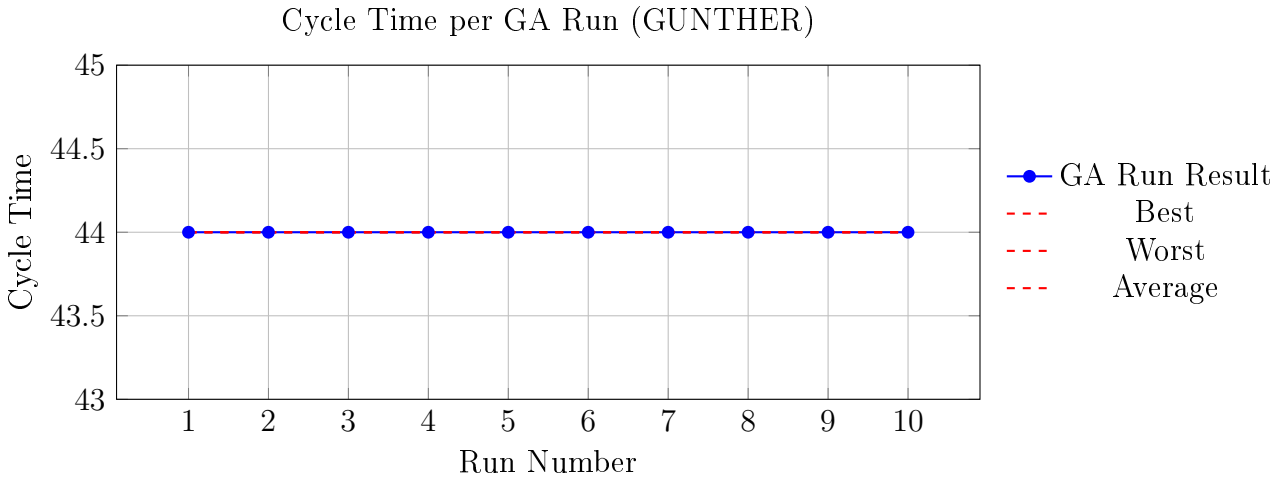

Figure 24: Instance GUNTHER cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

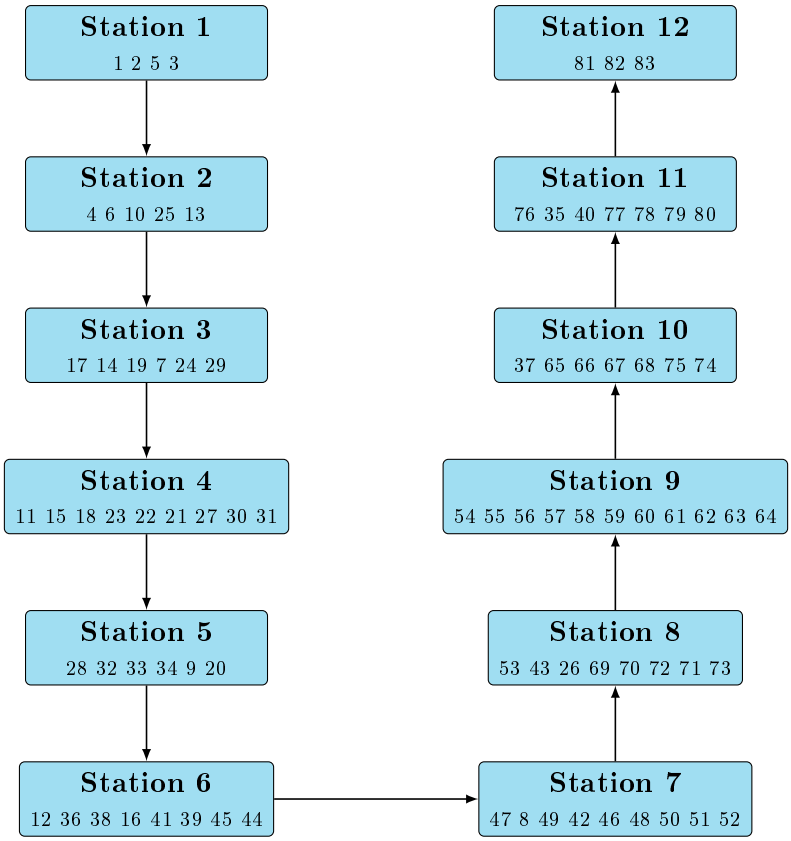

Figure 17: U-shaped 12-station assignment for ARC83 (Individual 7, Fitness = 0.000155).

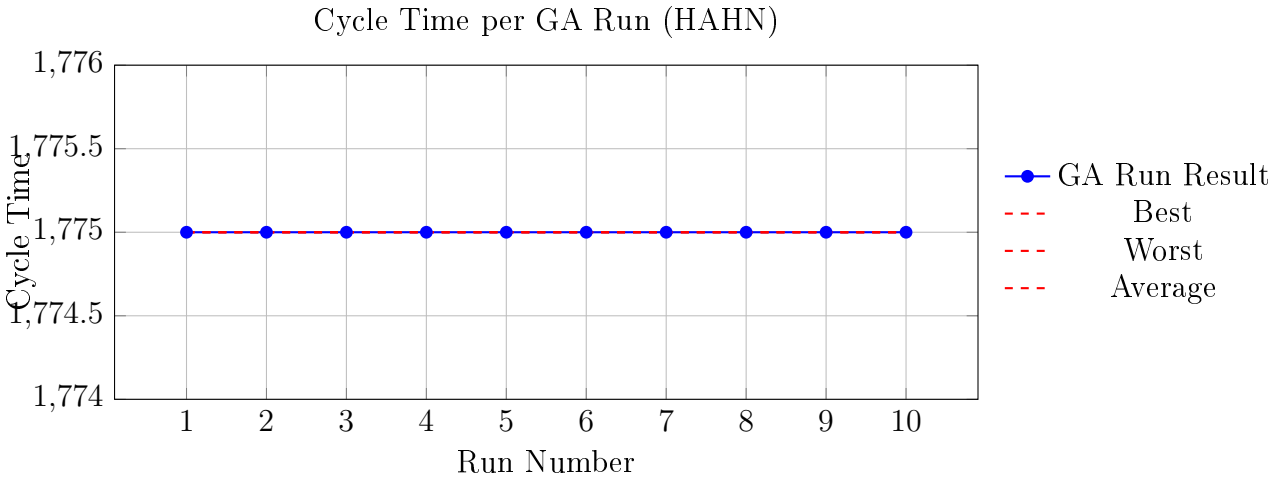

Figure 25: Instance HAHN cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

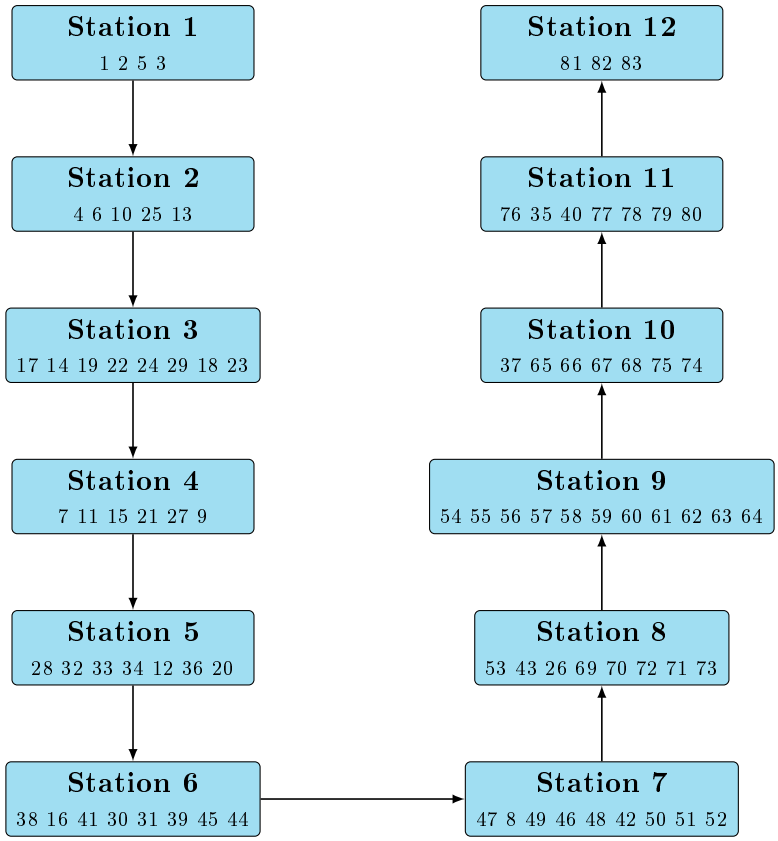

Figure 18: U-shaped 12-station assignment for ARC83 (Individual 8, Fitness = 0.000155).

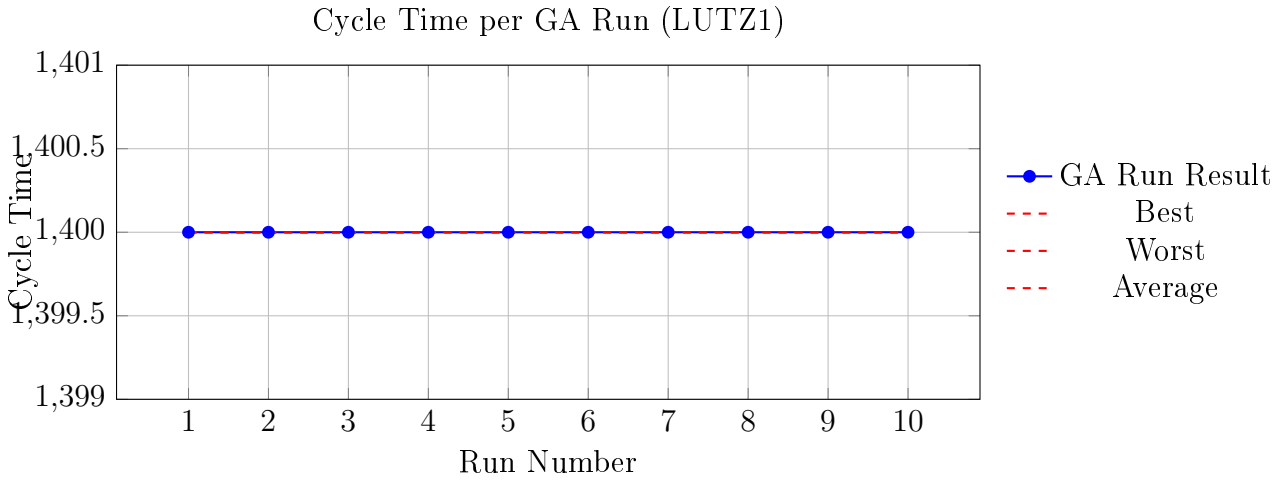

Figure 26: Instance LUTZ1 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

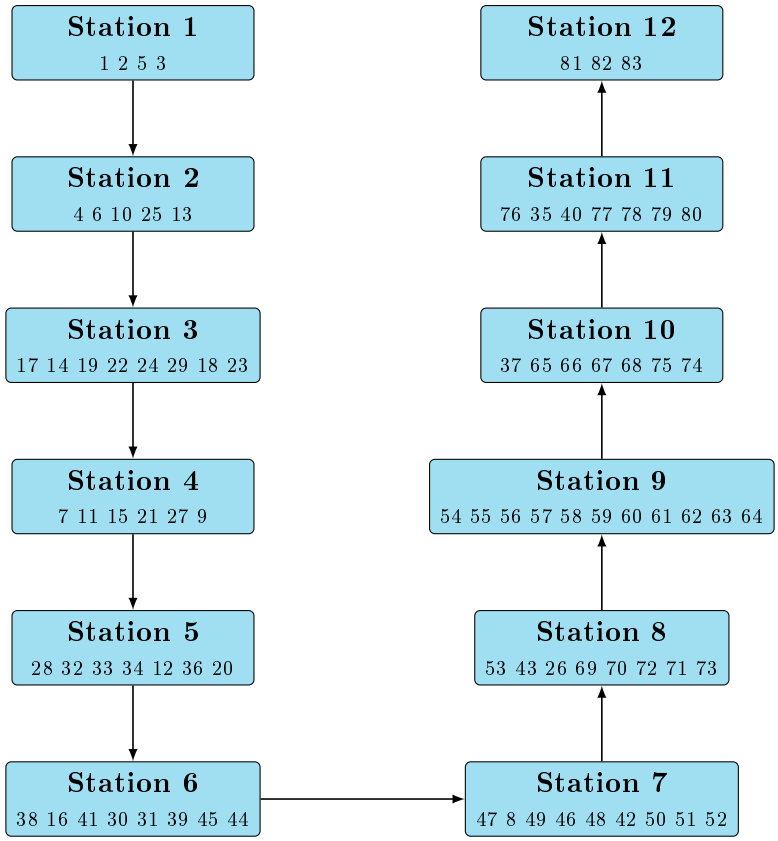

Figure 19: U-shaped 12-station assignment for ARC83 (Individual 9, Fitness = 0.000155).

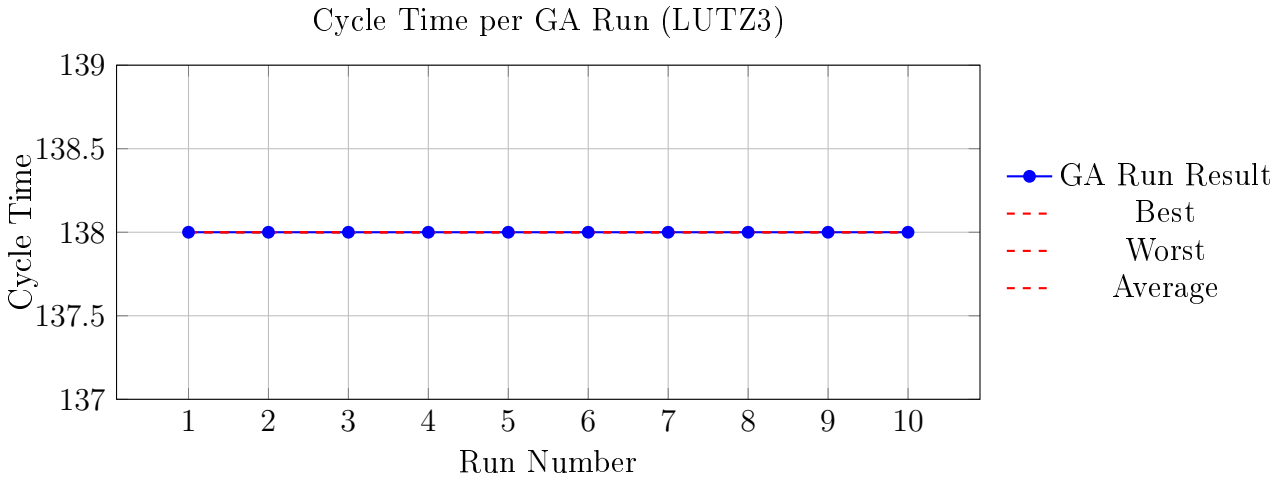

Figure 27: Instance LUTZ3 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

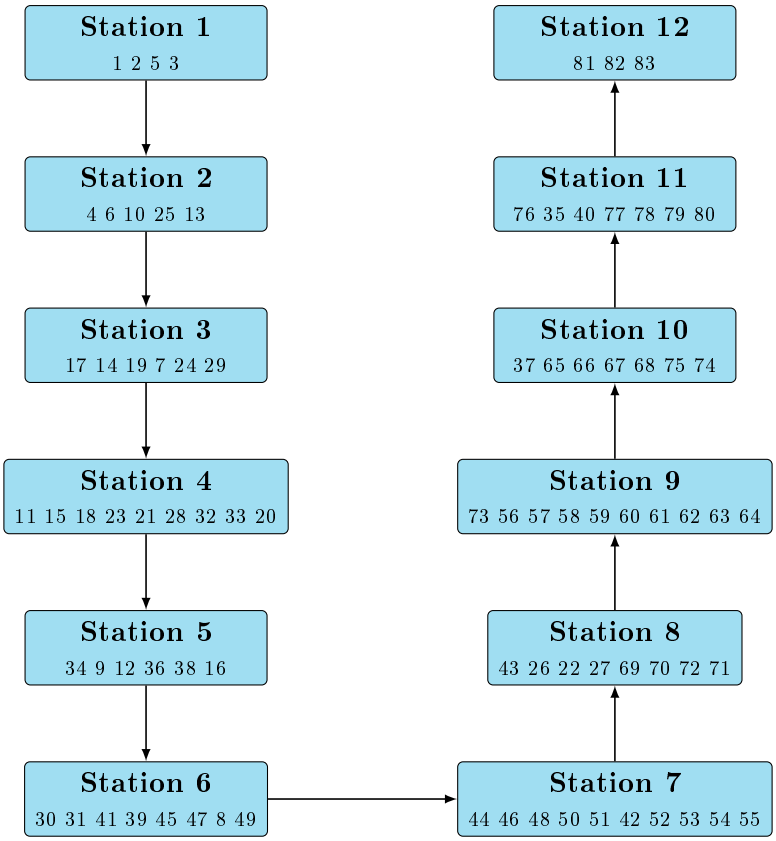

Figure 20: U-shaped 12-station assignment for ARC83 (Individual 10, Fitness = 0.000154).

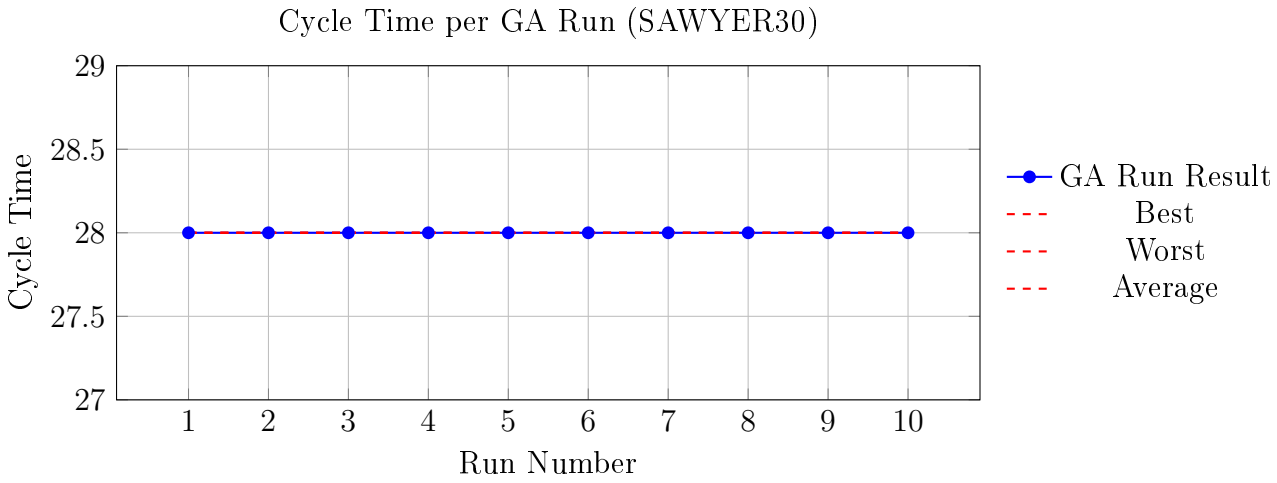

Figure 28: Instance SAWYER30 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

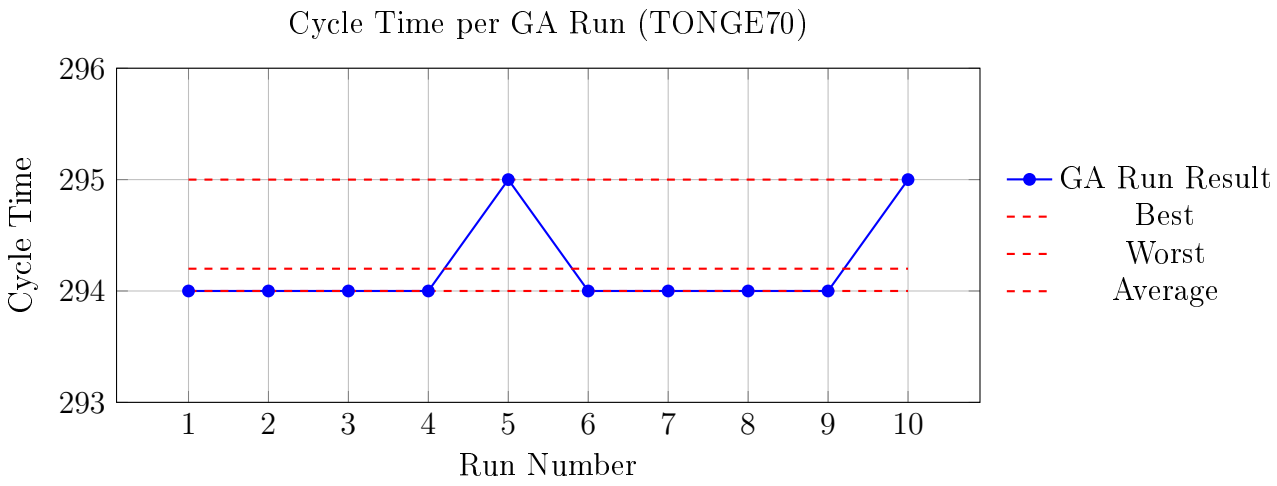

Figure 29: Instance TONGE70 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.